

CS336 作业 2

系统与并行

Systems and Parallelism · 中文全文翻译

版本 26.1.3 · CS336 课程组 · 2026 年春季

- 性能分析与混合精度
- 激活检查点与单 GPU 显存
- FlashAttention-2 与 Triton
- DDP、优化器状态分片与 FSDP
- 数据并行、张量并行与二维并行分析

本译稿完整保留题目编号、交付要求、代码接口、命令、变量名、测试名和网址；代码主体保持原样，说明文字译为中文。它是学习用中文翻译，不包含任何题目的答案。

1 作业概述

在本次作业中，你将通过实践学习如何提高单 GPU 的训练速度，以及如何把训练扩展到多块 GPU。

你将实现的内容

1. 基准测试与性能分析框架
2. 激活检查点 (activation checkpointing)
3. FlashAttention-2 Triton 内核
4. 分布式数据并行训练
5. 优化器状态分片
6. 完全分片数据并行训练

代码结构

本作业代码和本文档发布在 GitHub：

github.com/stanford-cs336/assignment2-systems

请使用 Git 克隆该仓库。如果后续有更新，课程组会通知你；届时可以通过 `git pull` 获取最新版。

1. `cs336-basics/`：本作业会分析作业 1 中实现的一些组件。此文件夹包含作业 1 的课程组参考实现，因此你会在其中看到 `cs336-basics/pyproject.toml` 和 `cs336-basics/cs336_basics` 包。如果想使用自己的模型实现，可以修改根目录中的 `pyproject.toml`，令其指向你自己的包。
2. `/`： `cs336-systems` 的根目录。课程组创建了一个名为 `cs336_systems` 的空模块。其中没有任何代码，因此你可以完全从零开始组织实现。
3. `tests/*.py`：这里包含你必须通过的全部测试。这些测试会调用 `tests/adapters.py` 中定义的接口钩子。你需要实现这些 adapter，把自己的代码连接到测试。为了调试，你可以编写更多测试或修改测试代码，但最终实现必须通过课程组提供的原始测试套件。
4. `README.md`：包含预期目录结构的更多细节，以及环境配置的基本说明。

提交方式

你需要在 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请使用排版工具整理答案。
- `code.zip`：包含你编写的全部代码。

运行 `test_and_make_submission.sh` 脚本生成 `code.zip`。

2 性能分析与基准测试

在作业第一部分，我们将研究如何优化 Transformer 模型的性能，从而更高效地利用 GPU。首先分析模型在前向传播和反向传播期间把时间与显存花在了哪里；然后用自定义 GPU 内核优化自注意力，使其性能超过常规 PyTorch 实现。之后，我们会使用多块 GPU，并理解如何在一个计算集群上训练模型。

2.1 性能分析

在动手优化之前，最好先分析程序，弄清时间、内存等资源消耗在什么地方。否则，我们可能优化了一个对总时间或总显存贡献很小的部分，最终看不到可测量的端到端收益。

我们将实现三条性能评估路径：

1. 使用 Python 标准库进行简单的端到端基准测试，为前向传播和反向传播计时；
2. 使用 NVIDIA Nsight Systems 做计算性能分析，了解时间如何分布在 CPU 和 GPU 的各项操作上；
3. 显存分析。

2.1.1 环境设置：导入基础 Transformer 模型

首先确认你能够加载上一份作业中的模型。作业 1 已经把模型组织成 Python 包，便于之后直接导入。课程组在 `/cs336-basics` 文件夹中加入了模型参考实现，并已在 `pyproject.toml` 中指向该实现。像往常一样调用 `uv run [command]` 时，`uv` 会自动找到本地 `cs336-basics` 包。如果想使用你自己的模型实现，可以修改 `pyproject.toml`，使其指向你的包。

可用下面的方式测试导入：

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
  Built cs336-systems @ file:///path/to/systems/dir
  Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr 7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

现在，作业 1 中的相关模块应该都可以使用。例如，对于 `model.py`，可以执行 `import cs336_basics.model`。

2.1.2 模型规模

本作业会对不同模型进行基准测试和性能分析，以理解模型规模变化时性能如何变化。除排行榜任务外，所有模型统一使用词表大小 10,000、批大小 4，并改变上下文长度。本次及后续作业需要用大量表格和图展示结果。强烈建议你用代码自动生成作业报告中的表格，因为在 LaTeX 或 Typst 中手工排表会很繁琐。可以参考 `pandas.DataFrame.to_latex()`、`pandas.DataFrame.to_typst()`，也可以从自己喜欢的表格数据结构出发编写生成函数。

规模	<code>d_model</code>	<code>d_ff</code>	<code>num_layers</code>	<code>num_heads</code>
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

表 1：不同模型规模的配置。它们大体基于 GPT-2 配置。

除非另有说明，统一使用 512 的上下文长度。

2.1.3 端到端基准测试

现在实现一个简单的性能评估脚本。我们将测试模型的许多变体，例如改变精度、替换层等，因此最好通过命令行参数控制这些选项，方便之后反复运行。

先做最简单的性能分析：分别测量模型前向传播、反向传播和优化器更新所需的时间。因为这里只测速度和显存，所以可以使用随机权重与随机数据。

性能测量并不简单，一些常见陷阱会让结果偏离真正想测的内容。对 GPU 代码做基准测试时，要特别注意 CUDA 调用是异步的。例如，调用 `torch.matmul` 触发 CUDA 内核后，PyTorch 函数不必等待矩阵乘法真正完成，就会把控制权交还给 Python 代码。这样 CPU 可以在 GPU 执行矩阵乘法的同时继续向前运行并调度新操作，这是重要的性能优势。

但这也意味着，直接测量 `torch.matmul` 函数返回用了多久，并不能告诉我们 GPU 实际执行矩阵乘法花了多久。在 PyTorch 中，可以调用 `torch.cuda.synchronize()`，等待已经调度的 GPU 内核全部完成，从而更准确地测量 CUDA 内核运行时间。这里的“同步”指 CPU 运行时与 GPU 运行时之间的同步。基于这一点，下面开始构建基础性能分析框架。

题目 benchmarking_script：基准测试脚本（4 分）

(a) 编写一个脚本，对模型的前向传播、反向传播和优化器更新做基本的端到端基准测试。脚本应支持：

- 根据给定超参数（例如层数）初始化模型；
- 生成一批随机数据；
- 先运行 w 个预热步骤，再对 n 个步骤计时。根据参数，计时对象可以是仅前向传播、前向加反向传播，或前向加反向传播再加优化器更新；
- 计时可使用 Python 的 `timeit` 模块，例如 `timeit` 函数或 `timeit.default_timer()`。后者使用系统最高精度时钟，因此比 `time.time()` 更适合作为基准测试的默认计时器；
- 每一步之后调用 `torch.cuda.synchronize()`。

提交内容： 一个脚本：根据给定超参数初始化基础 Transformer 模型、创建随机数据 batch，并分别测量仅前向、前向加反向、以及包含优化器更新的完整训练步骤。

(b) 对 2.1.2 节列出的模型规模，测量前向传播、反向传播和优化器更新。使用 5 个预热步骤，并在 10 个测量步骤上计算平均值和标准差。一次前向传播需要多久？反向传播呢？不同测量之间波动很大，还是标准差很小？

提交内容： 用 1-2 句话报告计时结果。

(c) 基准测试的一个常见问题是不做预热。取消预热并重复分析。结果受到怎样的影响？你认为原因是什么？再尝试只做 1 或 2 个预热步骤；为什么结果仍可能不同？

提交内容： 2-3 句话。

2.1.4 Nsight Systems 性能分析器

端到端基准测试只能告诉我们前向与反向传播总共花了多久、用了多少内存，却不能说明开销具体发生在哪里，因此也无法直接揭示优化机会。若想知道程序在每个组件（例如函数）中花费的时间，可以使用性能分析器。执行性能分析器会在函数开始和结束时插入监测点，从而给出函数级的详细执行统计，例如调用次数、平均耗时和累计耗时。

标准 Python 性能分析器（例如 CProfile）无法分析 CUDA 内核，因为这些内核在 GPU 上异步执行。好在 NVIDIA 提供了一个可通过命令行工具 `nsys` 使用的性能分析器。建议通过包管理器或官网下载页的安装程序获取最新版本。本节将用 `nsys` 分析 Transformer 模型的运行时间。

使用方法很直接：在上一节的 Python 脚本命令前加上 `nsys profile`。例如，对 `benchmark.py` 做基本分析：

```
$ uv run nsys profile -- python benchmark.py
```

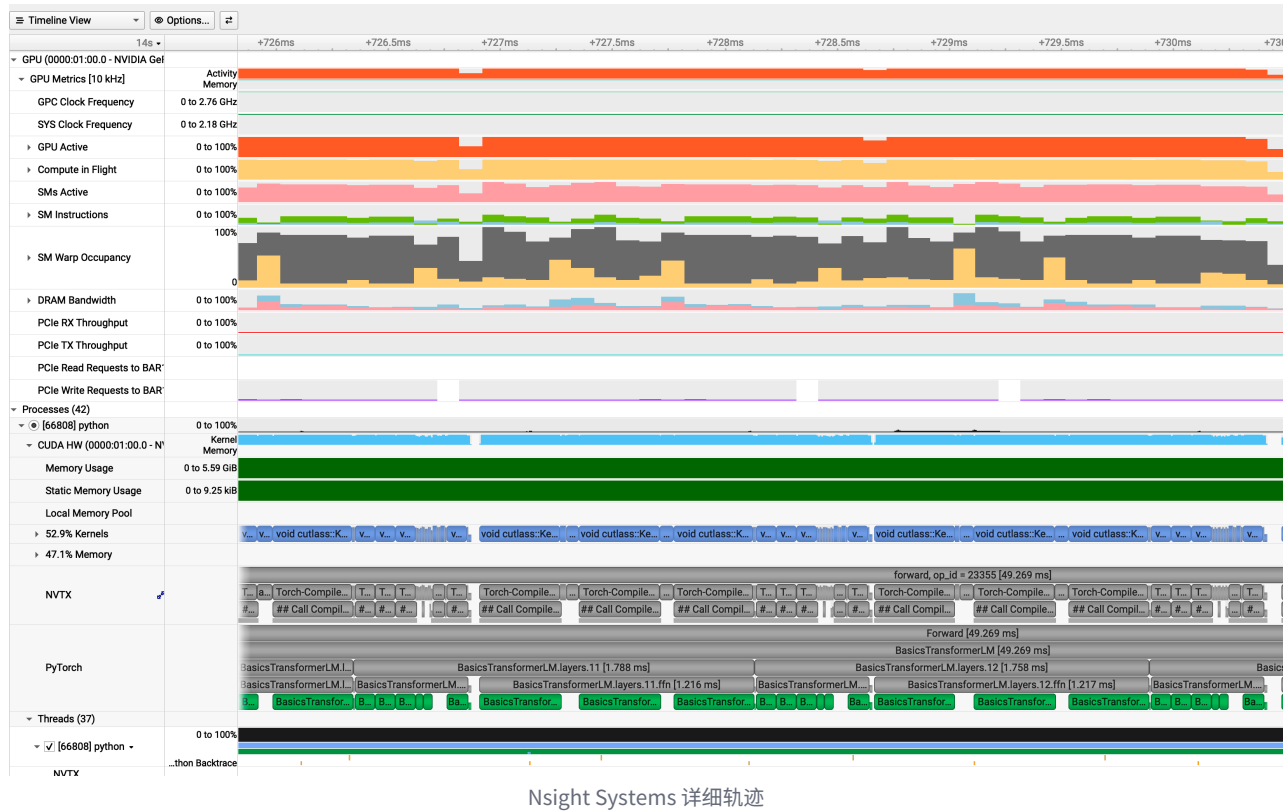
随后可以在本地使用 NVIDIA Nsight Systems 桌面应用打开结果。在性能轨迹的 `CUDA API` 行选择某个 CPU 侧 CUDA API 调用时，`CUDA HW` 行中对应的全部 GPU 内核执行会被高亮。

一次更全面的分析可以写成：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx \
--pytorch=functions-trace,autograd-shapes-nvtx \
```

```
--cudabacktrace=all --python-backtrace=cuda \
--gpu-metrics-devices=0 -- python benchmark.py
```

图 1：一条详细的 Nsight Systems 轨迹。



Nsight Systems 详细轨迹

这里，`--trace` 指定要记录哪些 API；`--pytorch` 在模块调用和 autograd 过程中插入 NVTX 标签；`--cudabacktrace` 与 `--python-backtrace` 提供更完整的回溯，帮助你理解某个内核由代码中的什么位置触发；`--gpu-metrics-devices` 指定要测量哪块 GPU 的利用率。

性能分析不是免费的，它会拖慢整体运行。一次运行通常只需开启当前真正要观察的功能。尤其在不需要调用栈时，可以移除 `--cudabacktrace=all` 和 `--python-backtrace=cuda`，因为二者的额外开销尤其大。

建议尝试 `nsys profile` 的不同命令行选项，熟悉它能提供的功能。你还可以用 NVTX range 给代码添加标注；这些范围会在性能轨迹的 NVTX 行显示为区块，其中包含全部 CUDA API 调用及其关联的内核执行。特别是，你应使用 NVTX range 从基准测试中排除预热步骤，即在性能分析时对相应 NVTX 标签使用 `--nvtx-capture` 过滤器。

你还可以分离出模型前向与反向传播分别使用了哪些内核，甚至可以通过下面的方式给实现添加标注，进一步区分自注意力层各部分所使用的内核：

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
):
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection

    return ...
```

在基准测试脚本中，可以这样用标注后的版本替换原实现：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

最后要注意, `torch.compile` 可能让时间和资源很难归因到源代码的具体部分。为了准确定位资源开销, 你很可能需要反复调整 `torch.compile` 和 NVTX 标注所包裹的代码范围。

题目 `nsys_profile`: Nsight Systems 性能分析 (5 分)

使用 `nsys` 分析前向传播、反向传播和优化器更新。请从表 1 中任选两个模型规模, 并使用三个大于 128 的、2 的幂次上下文长度; 其中最大值应是你显存中能容纳的最长上下文。选择你认为最值得观察的组合。对每份性能轨迹回答下列问题:

(a) 前向传播总共花了多少时间? 是否与之前用 Python 标准库测得的时间一致?

提交内容: 1-2 句话。

(b) 前向传播中, 哪一个 CUDA 内核累计占用的 GPU 时间最多? 模型完成一次前向传播会调用它多少次? 在同时执行前向和反向传播时, 占用时间最多的还是同一个内核吗?

提示: 查看 `Stats System View` 下的 `CUDA GPU Kernel Summary`, 并使用 NVTX range 过滤, 识别模型的哪些部分对应哪些内核。

提交内容: 1-2 句话。

(c) 虽然绝大部分 FLOPs 来自矩阵乘法, 但你会发现其他一些内核仍占据了不可忽略的总运行时间。前向传播中, 除矩阵乘法外, 还有哪些内核占用了明显的 CUDA 运行时间?

提交内容: 1-2 句话。

(d) 使用你自己实现的 AdamW, 分析一个完整训练步骤: 前向传播、计算损失并反向传播, 最后执行优化器更新。与仅做推理 (只有前向传播) 相比, 矩阵乘法所占时间比例如何变化? 其他内核呢?

提交内容: 1-2 句话。

(e) 比较模型前向传播时, 自注意力层中 softmax 操作与矩阵乘法操作的运行时间。这两者的运行时间差异, 与它们的 FLOPs 差异相比如何?

提交内容: 1-2 句话。

2.1.5 混合精度

到目前为止, 本作业一直使用 FP32 精度运行, 即模型参数和激活的类型都是 `torch.float32`。现代 NVIDIA GPU 包含专门用于加速低精度矩阵乘法的 Tensor Core。例如, NVIDIA B200 规格表显示, 其 FP32 峰值吞吐量为 80 TFLOPS, 而 FP16 (半精度浮点数) 或 BF16 (bfloat16) 的峰值吞吐量高达 2500 TFLOPS。因此, 使用更低精度的数据类型应能加速训练和推理。

不过, 简单粗暴地把模型转换为低精度格式, 可能会降低模型精度。例如, 实践中的许多梯度值太小, 无法用 FP16 表示, 若直接以 FP16 训练就会变成 0。为了缓解这个问题, FP16 训练通常使用损失缩放: 把损失乘上一个缩放因子, 增大梯度幅值, 避免梯度下溢为 0。此外, FP16 的动态范围小于 FP32, 容易发生溢出, 并表现为损失变成 NaN。全 BF16 训练通常更稳定, 因为 BF16 与 FP32 的动态范围相同, 但最终模型性能仍可能与 FP32 不同。

为了获得低精度数据类型带来的加速, 通常会使用混合精度训练。在 PyTorch 中, 这通过 `torch.autocast` 上下文管理器实现: 某些操作 (例如矩阵乘法) 使用低精度数据类型, 而需要 FP32 完整动态范围的操作 (例如累加与归约) 保持原精度。例如, 下面的代码会自动判断前向传播中的哪些操作应以低精度执行, 并把这些操作转换为指定类型:

```
model: torch.nn.Module = ... # e.g. your Transformer model
dtype: torch.dtype = ...     # e.g. torch.bfloat16
x: torch.Tensor = ...        # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

如上所述, 即使被累加的张量本身已经降为低精度, 通常也应让累加操作保持更高精度。下面的练习会帮助你直观理解原因。

题目 mixed_precision_accumulation: 混合精度累加 (1 分)

运行下面的代码，并评价结果的准确性。

```
s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float32)
print(s)

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01, dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

提交内容: 2-3 句话。

接下来，先把混合精度应用到一个玩具模型中以建立直觉，再把它应用到基准测试脚本。

题目 benchmarking_mixed_precision: 混合精度基准测试 (2 分)

(a) 考虑下面的模型：

```
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

假设在 GPU 上训练该模型，且模型参数最初为 FP32。现在希望通过 autocast 使用 FP16 混合精度。下列各项的数据类型分别是什么？

- autocast 上下文中的模型参数；
- 第一个前馈层 `ToyModel.fc1` 的输出；
- LayerNorm `ToyModel.ln` 的输出；
- 模型预测 logits；
- 损失；
- 模型梯度。

提交内容: 列出以上每个组件的数据类型。

(b) 你应当已经观察到，FP16 混合精度 autocast 对 LayerNorm 和前馈层的处理不同。LayerNorm 的哪些部分对混合精度敏感？如果使用 BF16 而非 FP16，是否仍需特殊处理 LayerNorm？为什么？

提交内容: 2-3 句话。

(c) 修改基准测试脚本，使其可以选择用 BF16 混合精度运行模型。对 2.1.2 节列出的每一种语言模型规模，分别测量启用与不启用混合精度时的前向和反向传播时间。比较全精度与混合精度结果，并讨论模型规模变化时呈现的趋势。这里可能会用到空操作上下文管理器 `nullcontext`。

提交内容： 用 2-3 句话报告计时结果并作出说明。

2.1.6 显存分析

前面一直关注计算性能。现在把注意力转向显存--语言模型训练和推理中的另一项主要资源。PyTorch 同样提供了强大的显存分析器，可以持续记录内存分配情况。

可以这样修改基准测试脚本来使用显存分析器：

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

上述代码会生成 `memory_snapshot.pickle`，可加载到在线工具 pytorch.org/memory_viz。在浏览器中打开该网页，把 pickle 文件拖入页面，即可查看总显存使用时间线，也能看到每一次内存分配的大小，以及追溯到分配位置的调用栈。

接下来使用 PyTorch 分析器研究模型的显存使用。

题目 `memory_profiling`：显存分析（4 分）

对表 1 中的 `xl` 模型执行完整训练步骤--前向传播、反向传播和优化器更新--并分别使用 128 与 2048 的上下文长度进行分析。

(a) 为性能分析脚本增加一个选项，使模型通过显存分析器运行。可以复用之前的基础设施，例如启用混合精度、加载指定规模的模型等。然后运行脚本，获取 `xl` 模型仅推理（只有前向传播）和完整训练步骤的显存轨迹。显存时间线呈现怎样的形态？能否根据峰值判断当时处于哪个阶段？

提交内容： 从 `memory_viz` 工具导出两张 `Active memory timeline` 图片：一张对应 `xl` 模型前向传播，另一张对应完整训练步骤（前向、反向、优化器更新）；再用 2-3 句话说明。

(b) 仅执行前向传播时，每个上下文长度的峰值显存是多少？执行完整训练步骤时呢？

提交内容： 一张表，每个上下文长度给出两个数值。

(c) 在混合精度下，分别测量 `xl` 模型前向传播和完整训练步骤的峰值显存。混合精度是否会显著影响显存使用？

提交内容： 2-3 句话。

(d) 考虑 `xl` 模型。使用给定参考超参数和单精度时，Transformer 残差流中的一个激活张量有多大？用 MiB 表示，即把字节数除以 1024^2 。

提交内容： 用 1-2 句话给出推导。

(e) 仔细观察 `xl` 模型做一次前向传播时，在 pytorch.org/memory_viz 中显示的 `Active Memory Timeline`。降低 `Detail` 级别后，该工具会隐藏相应比例的较小内存分配；例如，设为 10% 时只显示最大的 10% 分配。图中最大的内存分配有多大？查看调用栈后，能否判断这些分配来自哪里？

提交内容： 1-2 句话。

(f) Nsight Systems 也提供显存分析相关参数。可以把它们与前面的 Nsight 参数组合，理解模型生命周期不同阶段发生了哪些内存分配。使用 PyTorch 提供的 `NVTX` 标签，判断模型中单个 `TransformerBlock` 为反向传播保存了多少显存。这类张量通常称为 `residual`（残留张量）或 `saved tensor`（保存张量）。列出贡献最大的 5 个操作及其占总显存的百分比。

反向传播时，这些张量都会被释放，但同时会产生新的梯度张量。根据性能轨迹中前向传播分配的显存，以及反向传播中每经过一个 `TransformerBlock` 时显存的变化，计算一个 `TransformerBlock` 生成的梯度张量占用多少显存。结果是否符合预期？

提交内容： Nsight Systems 截图，以及 1-2 段文字说明。

3 单 GPU 显存

本作业后面的部分将研究如何把张量分片到多块 GPU 上。不过，即便只使用一块 GPU，也有一些可以节省显存的技巧，其中最常见的是梯度检查点，也称激活检查点。

3.1 Autograd 保存张量

回忆一下：为了完成模型的反向传播，必须保存前向传播产生的某些激活。对部分操作而言，这一点显而易见；但默认情况下，实际保存的张量往往比你预想的更多。为反向传播保存的张量称为 `residual`，也可以直接称为 `saved tensor`。

下面观察网络中究竟保存了什么。以一个看似普通的 `RMSNorm` 函数为例--为了简化，完全使用 FP32--并在 `autograd` 保存或取回张量时添加钩子：

```
import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)

class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x

def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

输出显示保存的张量多得令人担忧，其中好几个都和完整激活张量一样大：

```
$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32,
grad_fn=<RsqrtBackward0 object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32,
grad_fn=<RsqrtBackward0 object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32,
grad_fn=<RsqrtBackward0 object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

```

Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32,
grad_fn=<RsqrtBackward0 object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None

```

3.1.1 算子融合

在这个例子中，操作粒度显然过细。我们希望有一个单一算子：输入 RMSNorm 权重和激活，直接输出最终结果，并且在反向传播中也把它作为一个整体操作。内核融合的动机之一就在这里。RMSNorm 的计算形式相对规整，甚至可以直接通过 `torch.compile` 自动融合：

```

...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()

```

新的输出明显更好：

```

Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None

```

现在反向传播只需保存一个完整大小的激活张量，也就是 RMSNorm 函数的输入。还要注意，取回顺序不再是保存顺序的逆序，并且每个保存张量都不再带有 `grad_fn` 依赖。这说明 PyTorch 已把整个 RMSNorm 视为一个单独函数。

3.2 激活检查点

算子融合当然很有用，但它能节省的显存终究有限。例如，下面对 `xl` 规模的一个 `TransformerBlock` 做尽可能充分的融合：

```

import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(
    d_model=d_model,
    d_ff=d_ff,
    num_heads=num_heads,
    positional_encoder=RotaryEmbedding(
        dim=d_model // num_heads,
        context_length=context_length,
    ),
)

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)

...
# Now logs the number of bytes saved
total_size_bytes = 0

def pack_hook(t):
    if isinstance(t, torch.nn.Parameter): # Skip parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(
    "Total size of saved tensors in single TransformerBlock: "
    f"{total_size_bytes / (1024**2):.2f} MiB"
)

```

脚本显示，为反向传播保存了如下数量的显存：

```
...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB
```

每一层需要 3.6 GiB。如果所有层都这样做，仅为反向传播保存的激活就达到 114 GiB！注意力操作保存的张量中有相当一部分浪费，我们会在第 4 节修复；但即便修复后，显存使用仍会随 batch size、sequence length 和 embedding size 线性增长。

3.2.1 重计算

与其保留生成的每一个张量，我们可以只周期性保存结果检查点，并在需要时重新计算中间值。PyTorch 提供了简单接口 `torch.utils.checkpoint.checkpoint`：它接收一个函数以及传给该函数的参数，并修改该函数的执行行为。

1. 在前向传播中：

1. 保存传入该函数的输入值； 2. 禁止保存函数前向传播过程中产生的张量。

2. 在反向传播中：

1. 先执行一次重计算：从之前保存的输入重新运行前向传播，并为反向传播保存所需值； 2. 再运行反向传播，并在完成后释放全部张量。

在连续执行 4 个 Transformer block 的简单情形中，显存用量会按预期相加：

```
...
def four_blocks(x):
    x = block(x)
    x = block(x)
    x = block(x)
    x = block(x)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(
    "Total size of saved tensors in four TransformerBlocks: "
    f"{total_size_bytes / (1024**2):.2f} MiB"
)

Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

可以这样使用梯度检查点：

```
from torch.utils.checkpoint import checkpoint

def two_blocks(x):
    x = block(x)
    x = block(x)
    return x

def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass.
    # When reaching a checkpointed block during backward, it reruns forward
    # to produce saved tensors, then completes the normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(
    "Total size of saved tensors in four TransformerBlocks with checkpointing: "
    f"{total_size_bytes / (1024**2):.2f} MiB"
)

Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB
```

请注意，这并没有消除显存使用，而是把显存分成两类：第一类是在每次 `checkpoint` 调用的入口处，为之后重计算长期保存的检查点；第二类是在检查点区块内部重计算时，为完成该区块反向传播而临时生成的短期张量。

由于主要关心的是峰值显存，我们需要在“保存检查点的显存成本”与“物化一个完整区块 residual 的显存成本”之间取得平衡。检查点越多，一个区块内需要同时物化的显存越少，但检查点自身需要的显存越多。

在上面的例子中，重计算 residual 所需显存显然占主导地位，部分原因是检查点本身很小。因此，把平衡进一步移向“多保存一些检查点”会更有利，也就是缩小每个检查点区块的范围。

增大或缩小检查点区块，并不会改变一次普通重计算的总计算成本。如果愿意用更多计算换更低显存，还可以使用递归检查点：把 checkpoint 调用嵌套在其他 checkpoint 调用中。

题目 gradient_checkpointing: 显存最优的梯度检查点 (4 分)

考虑一个由 N 个相同 block 顺序堆叠的 Transformer。完全不使用检查点时，所有 N 个 block 的 residual 会同时存活，因此峰值激活显存是 $O(N)$ 。你可以自由选择前向传播的任意部分用 checkpoint 包裹，也可以嵌套检查点。

(a) 忽略计算成本，什么检查点策略能最小化峰值激活显存？说明如何安排 checkpoint 调用（可以给代码草图），并用 N 表示该策略的渐近峰值激活显存和计算量。假设单个 block 保存的 residual 远大于每个检查点自身的管理开销。

提交内容： 用 3-5 句话说明策略和渐近峰值显存，并附一小段代码草图。

(b) 考虑上述 `xl` 配置：batch size 为 4，sequence length 为 2048。如果你的时间/计算预算只允许一次重计算--也就是不能嵌套 checkpoint--最佳的检查点策略是什么？测量峰值显存以验证猜想。还要比较比所选策略更小和更大的相邻检查点区块大小，以确认结论。

提交内容： 用 3-5 句话说明推理，并给出所选策略的实测峰值显存。

4 GPU 内核

4.1 使用 FlashAttention-2 优化注意力

4.1.1 PyTorch 注意力基准测试

前面的性能分析很可能表明，注意力层在显存和计算方面都有优化空间。概括来说，注意力由一次矩阵乘法、softmax、再一次矩阵乘法构成：

$$\text{Attention}(Q, K, V) = \text{softmax}(\text{mask}(Q K^T / \sqrt{d_k})) V \quad (1)$$

朴素注意力实现必须为每一个 batch/head 元素保存形状为 $\text{seq_len} \times \text{seq_len}$ 的注意力分数矩阵。上下文很长时，该矩阵会变得极大，从而令长输入或长输出任务发生显存溢出。

我们将按照 FlashAttention-2 论文实现注意力内核。它以分块方式计算注意力，从始至终都不显式物化完整的 $\text{seq_len} \times \text{seq_len}$ 注意力分数矩阵，因此能够扩展到长得多的序列。

题目 pytorch_attention: PyTorch 注意力基准测试 (2 分)

(a) 在不同规模下测试你的注意力实现。编写脚本完成以下操作：

1. 固定 batch size 为 8，并且不使用多头注意力，即去掉 head 维度；
2. 遍历 head embedding 维度 $d_{\text{model}} \in [16, 32, 64, 128]$ 与 sequence length $\in [256, 1024, 4096, 8192, 16384]$ 的笛卡尔积；
3. 按相应大小随机创建输入 Q 、 K 、 V ；
4. 使用这些输入对注意力的 100 次前向传播计时；
5. 测量反向传播开始前已使用的显存，并为 100 次反向传播计时；
6. 务必先预热，并在每次前向/反向传播后调用 `torch.cuda.synchronize()`。

具体哪些配置会显存溢出取决于你的 GPU。报告各配置的运行时间或 OOM 错误。多大规模时开始 OOM？选择一个会 OOM 的最小配置，根据作业 1 的 Transformer 显存用量公式，详细核算注意力的显存占用。为反向传播保存的显存如何随 sequence length 变化？怎样消除这部分显存成本？

提交内容： 一张计时结果表、显存用量计算，以及 1-2 段文字回答。

4.2 JIT 编译注意力的基准测试

从 2.0 版本起，PyTorch 自带了强大的即时编译器，会尝试自动对 PyTorch 函数应用多种优化。简介见：

https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html

它会动态分析计算图，并尝试自动生成融合的 Triton 内核。PyTorch 编译器的接口很简单。例如，要编译模型中的单层，可以写：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

此后，`compiled_layer` 在功能上与 `layer` 一样，包括前向和反向传播。你也可以用 `torch.compile(model)` 编译整个 PyTorch 模型，甚至可以编译一个调用 PyTorch 操作的普通 Python 函数。

题目 torch_compile: Torch Compile (2 分)

(a) 扩展注意力基准测试脚本，加入 PyTorch 注意力实现的编译版本，并使用 `pytorch_attention` 题中的相同配置，把它的性能与未编译版本比较。

提交内容： 一张表，把编译版注意力模块的前向、反向传播耗时与 `pytorch_attention` 题中的未编译版本进行比较。

(b) 在端到端基准测试脚本中编译整个 Transformer 模型。前向传播性能如何变化？完整的前向、反向和优化器更新性能又如何变化？

提交内容： 一张表，对比原始 Transformer 与编译后的 Transformer。

从前面观察到的 sequence length 扩展行为来看，要处理大规模序列，我们需要显著改进。即使使用 `torch.compile`，当前实现面对长序列时仍有很差的内存访问模式。为了解决这一问题，接下来用 Triton 实现 FlashAttention-2，从而更精细地控制内存访问方式和计算时机。

4.2.1 示例：加权求和

为了介绍 Triton 的必要知识，以及它如何与 PyTorch 互操作，下面完整讲解一个“加权求和”操作的示例内核。若想进一步学习 Triton，可以参阅 Triton 官方教程。需要注意，那些教程没有使用较新的、很方便的 block pointer 抽象；本节会说明它的用法。

给定输入矩阵 X ，把它的每个元素乘以一个按列作用的权重向量 w ，再对每一行求和；所得结果就是 X 和 w 的矩阵-向量乘积。先分析该操作的前向传播，再为反向传播编写 Triton 内核。

前向传播

该内核的前向传播就是下面这个带广播的内积：

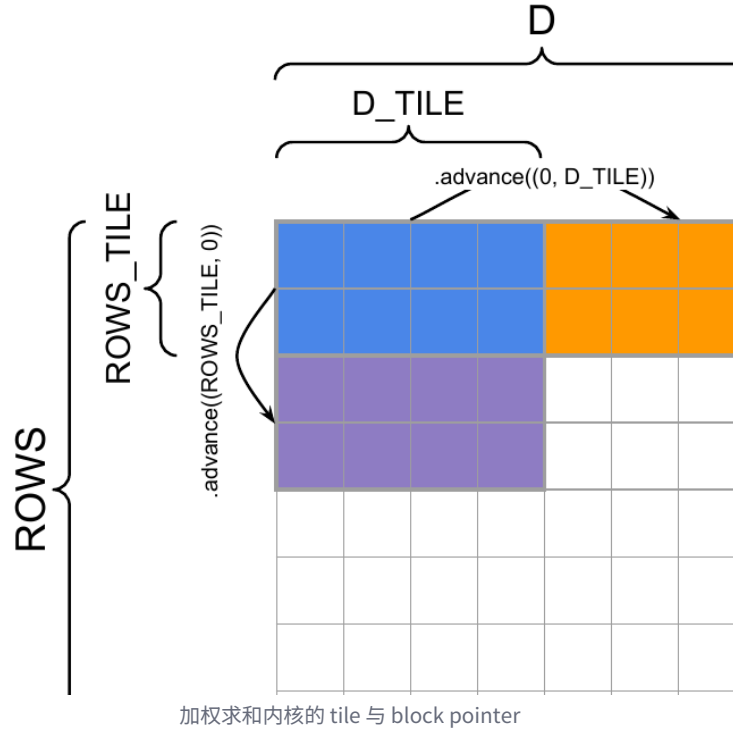
```
def weighted_sum(x, weight):
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]
    return (weight * x).sum(axis=-1)
```

在 Triton 内核中，每个 program instance（程序实例）可以并行运行，它负责计算 x 中一块行 tile 的加权和，并把对应的标量结果写入输出张量。Triton 的一个程序实例是一组执行同一程序的线程；这些线程块可以在 GPU 上并行运行。

内核不直接接收张量，而是接收指向张量首元素的指针，以及每个张量的 stride。stride 告诉我们沿张量各轴移动一个位置时，指针应移动多少。

利用 stride，可以加载当前实例负责求和的 x 行块。程序 ID 用来划分任务：实例 i 处理 x 的第 i 个行块。在这个简单示例中，Triton 前向传播与 PyTorch 实现的主要区别，就是必须显式完成指针运算和 load/store。下面使用 `tl.make_block_ptr` 的 block pointer 抽象，大幅简化指针运算，但在此之前要先为 block pointer 做一些设置。

图 2：加权求和内核中的分块方式，以及 block pointer 的推进方式。



上图示意了 tile 划分以及 block pointer 如何推进。前面的加权求和函数可以写成下面的 Triton 内核：

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr, # Input pointers
    output_ptr, # Output pointer
    x_stride_row, x_stride_dim, # Strides for moving one element on each axis
    weight_stride_dim, # Likely 1
    output_stride_row, # Likely 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr,
    D_TILE_SIZE: tl.constexpr, # Tile shapes known at compile time
):
    # Each instance computes the weighted sum of a tile of rows of x.
    # tl.program_id identifies the thread block currently running.
    row_tile_idx = tl.program_id(0)

    # A block pointer selects an ND memory region and moves that selection.
    # It needs:
    # - pointer to the tensor's first element
    # - overall tensor shape for out-of-bounds handling
    # - stride of every dimension
    # - ND coordinates of the starting block (offsets)
    # - block shape loaded/stored at one time
    # - memory order from major to minor; axes (= np.argsort(strides))
    # is needed for TMA support on Hopper and newer GPUs
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,),
        strides=(x_stride_row, x_stride_dim),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )
```

```

weight_block_ptr = tl.make_block_ptr(
    weight_ptr,
    shape=(D,),
    strides=(weight_stride_dim,),
    offsets=(0,),
    block_shape=(D_TILE_SIZE,),
    order=(0,),
)

output_block_ptr = tl.make_block_ptr(
    output_ptr,
    shape=(NUM_ROWS,),
    strides=(output_stride_row,),
    offsets=(row_tile_idx * ROWS_TILE_SIZE,),
    block_shape=(ROWS_TILE_SIZE,),
    order=(0,),
)

# Initialize a buffer to write to.
output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

for i in range(tl.cdiv(D, D_TILE_SIZE)):
    # Because the tile sizes may not divide the full dimensions,
    # both dimensions need boundary checks.
    row = tl.load(
        x_block_ptr,
        boundary_check=(0, 1),
        padding_option="zero",
    ) # (ROWS_TILE_SIZE, D_TILE_SIZE)
    weight = tl.load(
        weight_block_ptr,
        boundary_check=(0,),
        padding_option="zero",
    ) # (D_TILE_SIZE,)

    # Compute the weighted sum of each row.
    output += tl.sum(row * weight[None, :], axis=1)

    # Move pointers to the next tile. Deltas are (rows, columns).
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))

# Write one scalar per row; check the row boundary.
tl.store(output_block_ptr, output, boundary_check=(0,))

```

接下来将该内核包装进一个能与 PyTorch 互操作的 Autograd function: 它接收 Tensor, 输出 Tensor, 稍后也能接入 autograd 引擎完成反向传播。

```

class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight for the backward pass. During backward we only
        # receive the output gradient, and must compute gradients wrt inputs.
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D.
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)

        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

        # Roughly 16 loops through the embedding dimension.
        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16
        # Each thread block processes 16 batch elements at a time.
        ctx.ROWS_TILE_SIZE = 16
        ctx.input_shape = input_shape

        # Empty does not imply elements are initialized to zero.
        y = torch.empty(output_dims, device=x.device)

        # Launch n instances in a 1D grid.
        n_rows = y.numel()
        weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE))](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            NUM_ROWS=n_rows,
            D=D,

```

```

        ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE,
        D_TILE_SIZE=ctx.D_TILE_SIZE,
    )

    return y.view(input_shape[:-1])

```

调用 Triton 内核时，表达式

```
weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]
```

通过元组 `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)` 定义了线程块的 launch grid。随后可在内核内部用 `tl.program_id(0)` 取得线程块索引。

反向传播

因为定义了自己的内核，所以还需要自己编写 `backward` 函数。

前向传播接收层的输入并计算输出。反向传播则接收目标函数相对于输出的梯度，再计算相对于每个输入的梯度。这里，操作的输入为矩阵 $x \in \mathbb{R}^{(n \times h)}$ 和权重向量 $w \in \mathbb{R}^h$ 。简记该操作为 $f(x, w)$ ，其值域是 $\mathbb{R}^n$ 。若给定损失 L 相对于层输出的梯度 $\nabla f(x, w)L$ ，由多元链式法则可得：

$$(\nabla_x L)_{ij} = \sum_k [\partial f(x, w)_k / \partial x_{ij}] (\nabla f(x, w)L)_k = w_j \cdot (\nabla f(x, w)L)_i \quad (2)$$

$$(\nabla_w L)_j = \sum_i [\partial f(x, w)_i / \partial w_j] (\nabla f(x, w)L)_i = \sum_i x_{ij} \cdot (\nabla f(x, w)L)_i \quad (3)$$

因此，反向传播公式很简单。相对于 x 的梯度由公式 (2) 给出，即计算 w 与 $\nabla f(x, w)L$ 的外积；相对于 w 的梯度，即 $(\nabla_w L)_j$ ，则要把输入的对元素与输出梯度相乘并沿行求和。

反向传播内核先定义所有 block pointer，再计算 $\nabla_x L$ ：

```

@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr,          # Inputs
    grad_output_ptr,           # Input gradient from next layer
    grad_x_ptr, partial_grad_weight_ptr, # Output gradients
    stride_xr, stride_xd,
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr,
    D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS, D), strides=(stride_gr, ),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, ),
        block_shape=(ROWS_TILE_SIZE, ), order=(0, ),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D, ), strides=(stride_xr, stride_xd, ),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0, ),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE, ), order=(1, 0, ),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D, ), strides=(stride_wd, ),
        offsets=(0, ), block_shape=(D_TILE_SIZE, ), order=(0, ),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D, ), strides=(stride_gxr, stride_gxd, ),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0, ),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE, ), order=(1, 0, ),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,

```

```

    shape=(n_row_tiles, D), strides=(stride_gwb, stride_gwd),
    offsets=(row_tile_idx, 0),
    block_shape=(1, D_TILE_SIZE), order=(1, 0),
)

for i in range(tl.cdiv(D, D_TILE_SIZE)):
    grad_output = tl.load(
        grad_output_block_ptr,
        boundary_check=(0,), padding_option="zero",
    ) # (ROWS_TILE_SIZE,)

    # Outer product for grad_x.
    weight = tl.load(
        weight_block_ptr,
        boundary_check=(0,), padding_option="zero",
    ) # (D_TILE_SIZE,)
    grad_x_row = grad_output[:, None] * weight[None, :]
    tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

    # Reduce as many rows as possible for grad_weight.
    row = tl.load(
        x_block_ptr,
        boundary_check=(0, 1), padding_option="zero",
    ) # (ROWS_TILE_SIZE, D_TILE_SIZE)
    grad_weight_row = tl.sum(
        row * grad_output[:, None], axis=0, keep_dims=True
    )
    tl.store(
        partial_grad_weight_block_ptr,
        grad_weight_row,
        boundary_check=(1,),
    )

    # Advance pointers along D.
    x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
    weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
    partial_grad_weight_block_ptr = \
        partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
    grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))

```

计算 ∇x 很直接，结果可写入输出张量对应的 tile；计算 ∇w 则更棘手。每个内核实例只负责 x 的一个行块，但现在需要沿 x 的所有行求和。

我们不在该反向内核中直接完成整个求和，而是假设 `partial_grad_weight_ptr` 指向一个 `n_row_tiles × H` 矩阵。其第一维只在 x 当前行块内部完成归约，并把局部结果写入该张量。退出内核后，再用 `torch.sum` 把各行块结果相加，得到最终的 ∇w 。当然，也可以另写一个内核来完成该归约。

`autograd.Function` 的最后一部分相对简单：

```

class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE = ctx.ROWS_TILE_SIZE
        D_TILE_SIZE = ctx.D_TILE_SIZE
        n_rows, D = x.shape

        # Each block writes a partial buffer; reduce it afterward.
        partial_grad_weight = torch.empty(
            (triton.cdiv(n_rows, ROWS_TILE_SIZE), D),
            device=x.device,
            dtype=x.dtype,
        )
        grad_x = torch.empty_like(x)

        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows,
            D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE,

```

```

        D_TILE_SIZE=D_TILE_SIZE,
    )
    grad_weight = partial_grad_weight.sum(axis=0)
    return grad_x, grad_weight

```

最后得到一个行为很像 `torch.nn.functional` 中函数的接口：

```
f_weightedsum = WeightedSumFunc.apply
```

现在对两个 PyTorch 张量 `x` 和 `w` 调用 `f_weightedsum`，会得到类似下面的张量：

```

tensor([ 90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)

```

注意输出张量附带的 `grad_fn`：这表明，当该张量出现在计算图中时，PyTorch 知道反向传播应调用什么函数。至此，加权求和操作的 Triton 实现完成。

4.2.2 FlashAttention-2 前向传播

接下来按照 FlashAttention-2 (T. Dao, 2023) 实现一个性能显著优于普通 PyTorch 注意力的 Triton 版本。FlashAttention-2 使用若干技巧按 tile 计算前向传播，从而形成高效的内存访问模式，并避免在全局显存中物化完整的注意力矩阵。

进入本节前，强烈建议至少阅读最初的 FlashAttention 论文 (T. Dao 等, 2022)。它会帮助你理解高效注意力的核心技术：跨 tile 在线计算 softmax。该技术最早由 M. Milakov 等 (2018) 提出。若想更直观地理解 GPU 如何执行 PyTorch 代码，也建议阅读 H. He 的文章 [4]。

理解普通注意力的低效之处

忽略 mask 时，注意力前向传播可写为：

```

S = Q K^T / sqrt(d)                (4)
P_ij = softmax_j(S)_ij             (5)
O = P V                             (6)

```

标准反向传播为：

```

dV = P^T dO                        (7)
dP = dO V^T                        (8)
dS_i = dsoftmax(dP_i) = (diag(P_i) - P_i P_i^T) dP_i (9)
dQ = dS K / sqrt(d)                (10)
dK = dS^T Q / sqrt(d)              (11)

```

可以看到，反向传播依赖前向传播产生的一些巨大激活。例如，公式 (7) 计算 `dV` 需要 `P`。`P` 是形状为 `(batch_size, n_heads, seq_len, seq_len)` 的注意力分数，其大小随序列长度平方增长。这解释了前面在长序列注意力基准测试中遇到的显存问题。

普通注意力的前向和反向传播都必须在片上 SRAM 与 GPU HBM 之间传输 `P` 等大量量，付出高昂的内存 I/O 成本，而且某些张量还会重复传输。例如，标准反向传播在计算公式 (7) 和公式 (9) 时都会从 HBM 读取 `P`。

FlashAttention 的主要目标，就是避免注意力矩阵在 HBM 中反复读写，从而降低 I/O 与峰值显存。为此使用三项技术：分块、重计算和算子融合。

分块 (Tiling)

为了不把注意力矩阵写入 HBM 再读回，我们必须在无法一次访问完整输入的情况下完成 softmax 归约。具体方法是重新组织注意力计算：把输入拆成 tile，多次遍历输入 tile，并逐步完成 softmax 归约。

重计算 (Recomputation)

我们不在 HBM 中保存形状为 `(batch_size, n_heads, seq_len, seq_len)` 的巨大中间注意力矩阵。相反，只在 HBM 中保存某些“激活检查点”，并在反向传播时重算一部分前向传播，以重新获得计算梯度所需的其他激活。

FlashAttention-2 还会保存注意力分数的 `logsumexp` 值 `L`，用来简化反向传播。`L` 的表达式是：

```
L_i = log(sum_j exp(S_ij))          (12)
```

最终内核会在线计算它，但最终结果应与上式相同。结合分块与重计算后，内存 I/O 和峰值显存不再依赖 sequence_length^2 ，因此可以使用更长的序列。

算子融合 (Operator Fusion)

最后，把注意力涉及的全部操作放进单一内核，避免注意力矩阵及其他中间激活被反复传输。这称为算子融合或内核融合。我们将为前向传播编写一个 Triton 内核，在 HBM 与 SRAM 之间只做有限的数据传输，就完成全部注意力操作。

算子融合的一部分能力来自重计算：因为不必把每一个中间激活写回 HBM，所以能避免常规实现中的大量 I/O。关于这些技术的更多直观解释，可参阅 FlashAttention 系列论文 (T. Dao 等, 2022; T. Dao, 2023)。

使用重计算的反向传播

利用 L ，可以在反向传播中正确重计算，并高效求出梯度。在开始反向传播前，先在全局显存中预计算：

```
D = rowsum(0 * d0)
```

其中 $\circ$ 表示逐元素乘法。它等于 $\text{rowsum}(P \circ dP)$ ，因为：

```
P * dP^T = P * (d0 * V^T)^T = (P * V) * d0^T = 0 * d0^T
rowsum(A * B) = diag(A * B^T)
```

有了向量 L 和 D ，反向传播无需显式调用 softmax。完整计算如下：

```
S = Q * K^T / sqrt(d)           (13)
P_ij = exp(S_ij - L_i)          (14)
dV = P^T * d0                   (15)
dP = d0 * V^T                   (16)
dS_ij = P_ij * (dP_ij - D_i)    (17)
dQ = dS * K / sqrt(d)           (18)
dK = dS^T * Q / sqrt(d)         (19)
```

可以看到，这组操作不要求在前向传播时把注意力分数 P 保存在 HBM 中。反向传播会根据 Q 、 K 和 L ，通过公式 (13) 与 (14) 重新计算 P 。

FlashAttention 前向传播细节

理解了 FlashAttention-2 的核心技术后，下面深入你需要实现的 FA2 前向内核。

为了避免从 HBM 读取或向 HBM 写入注意力矩阵，我们希望通过分块让每一个输出 tile 可以独立计算。这要求能计算 P 的 tile，理想情况下同时沿 query 与 key 两个维度分块。

问题是：对 S 使用 softmax 时，需要归约 S 的完整一行才能得到分母，因此不能直接逐 tile 独立计算 P 。FlashAttention-2 用在线 softmax 解决这一问题。

下文用下标 i 表示当前 query tile，用上标 (j) 表示当前 key tile。query 维 tile 大小为 B_q ，key 维 tile 大小为 B_k ；hidden 维 d 不分块。

同时维护两个按行的运行值： $m_i^{(j)} \in \mathbb{R}^{(B_q)}$ 和 $l_i^{(j)} \in \mathbb{R}^{(B_q)}$ 。 $m_i^{(j)}$ 是运行最大值，用来数值稳定地计算 softmax；这正是作业 1 实现 softmax 时使用过的技巧。每处理一个新的 S 行块，即 j 增大时，都要更新 m_i 。

利用运行最大值，可计算未归一化的 softmax 值，也就是分子：

```
P_tilde_i^{(j)} = exp(S_i^{(j)} - m_i^{(j)})
```

$l_i^{(j)}$ 是 softmax 分母的运行代理量，按下面的规则更新：

```
l_i^{(j)} = exp(m_i^{(j-1)} - m_i^{(j)}) * l_i^{(j-1)}
          + rowsum(P_tilde_i^{(j)})
```

最终写回输出时，要用处理完全部 key tile 后 l_i 的最终值 $l_i^{(T_k)}$ 完成归一化。算法 1 展示了 GPU 上的前向传播。

算法 1: FlashAttention-2 前向传播

```

 $Q \in \mathbb{R}^{(N_q \times d)}$ ,  $K, V \in \mathbb{R}^{(N_k \times d)}$ ,  $\text{tile}$   $B_q \times B_k$ 

1.  $Q$   $\text{tile}$   $T_q = \lceil N_q / B_q \rceil$   $\text{tile}$   $Q_1, \dots, Q_{T_q}$ 
    $B_q \times d$ 
2.  $K, V$   $\text{tile}$   $T_k = \lceil N_k / B_k \rceil$   $\text{tile}$ 
    $K^{\wedge}(1), \dots, K^{\wedge}(T_k)$   $V^{\wedge}(1), \dots, V^{\wedge}(T_k)$   $B_k \times d$ 
3.  $i = 1, \dots, T_q$ 
   a.  $Q_i$ 
   b.  $O_i^{\wedge}(\theta) = \theta \in \mathbb{R}^{(B_q \times d)}$ 
       $L_i^{\wedge}(\theta) = \theta \in \mathbb{R}^{(B_q)}$   $m_i^{\wedge}(\theta) = -\infty \in \mathbb{R}^{(B_q)}$ 
   c.  $j = 1, \dots, T_k$ 
      i.  $K^{\wedge}(j) V^{\wedge}(j)$ 
      ii.  $\text{softmax}$   $\text{tile}$ 
           $S_i^{\wedge}(j) = Q_i (K^{\wedge}(j))^{\wedge T} / \sqrt{d} \in \mathbb{R}^{(B_q \times B_k)}$ 
      iii.  $m_i^{\wedge}(j) = \max(m_i^{\wedge}(j-1), \text{rowmax}(S_i^{\wedge}(j))) \in \mathbb{R}^{(B_q)}$ 
      iv.  $P_{\text{tilde}_i^{\wedge}}(j) = \exp(S_i^{\wedge}(j) - m_i^{\wedge}(j)) \in \mathbb{R}^{(B_q \times B_k)}$ 
      v.  $L_i^{\wedge}(j) = \exp(m_i^{\wedge}(j-1) - m_i^{\wedge}(j)) L_i^{\wedge}(j-1) + \text{rowsum}(P_{\text{tilde}_i^{\wedge}}(j)) \in \mathbb{R}^{(B_q)}$ 
      vi.  $O_i^{\wedge}(j) = \text{diag}(\exp(m_i^{\wedge}(j-1) - m_i^{\wedge}(j))) O_i^{\wedge}(j-1) + P_{\text{tilde}_i^{\wedge}}(j) V^{\wedge}(j)$ 
   d.  $O_i = \text{diag}((L_i^{\wedge}(T_k))^{\wedge(-1)}) O_i^{\wedge}(T_k)$ 
   e.  $L_i = m_i^{\wedge}(T_k) + \log(L_i^{\wedge}(T_k))$ 
   f.  $O_i$   $\text{tile}$   $O$   $i$   $\text{tile}$ 
   g.  $L_i$   $\text{tile}$   $L$   $i$   $\text{tile}$ 
4.  $O$   $\log\text{sumexp}$   $L$ 

```

在动手用 Triton 实现前向传播之前，下面汇总一些编写 Triton 内核的通用技巧。

Triton 技巧与注意事项

- 可在 Triton 中用 `tl.device_print` 打印调试信息：https://triton-lang.org/main/python-api/generated/triton.language.device_print.html。
- 设置 `TRITON_INTERPRET=1` 可以在 CPU 上运行 Triton 解释器，但课程组发现它存在一些 bug。
- 定义 block pointer 时，应确认 offset 正确，并且 block offset 已乘以相应的 tile size。
- `torch.autograd.Function` 子类的方法中，用下面的语法设置线程块 launch grid，正如加权求和示例所示：

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

- 使用 `tl.dot` 执行矩阵乘法。
- 推进 block pointer 时使用：`*_block_ptr = *_block_ptr.advance(...)`。

题目 flash_forward: FlashAttention-2 前向传播 (15 分)

(a) 编写一个纯 PyTorch（不使用 Triton）的 `autograd.Function`，实现 FlashAttention-2 前向传播。它会比常规 PyTorch 注意力慢得多，但能帮助你调试 Triton 内核。

实现应接收 Q 、 K 、 V 和标志 `is_causal`，并产生输出 O 与 `logsumexp` 值 L 。本小题可忽略 `is_causal` 标志。

`autograd.Function.forward` 应保存 L 、 Q 、 K 、 V 、 O 供反向传播使用，并返回 O 。注意，`autograd.Function` 的 `forward` 方法总是以 `context` 作为第一个参数。每个 `autograd.Function` 类都必须实现 `backward`，但目前可以只让它抛出 `NotImplementedError`。如需参考值，可以直接在 PyTorch 中实现公式 (4)-(6) 和 (12)，再比较输出。

接口应为：

```
def forward(ctx, Q, K, V, is_causal=False)
```

tile size 自行确定，但必须至少为 16×16 。测试中的各维度始终是 2 的幂且不小于 16，因此无需处理越界访问。

提交内容： 一个实现 FlashAttention-2 前向传播的 `torch.autograd.Function` 子类。为了测试，实现 `[adapters.get_flashattention_autograd_function_pytorch]`，再运行：

```
uv run pytest -k test_flash_forward_pass_pytorch
```

并确认测试通过。

(b) 按照算法 1，为 FlashAttention-2 前向传播编写 Triton 内核。再编写另一个 `torch.autograd.Function` 子类，在 `forward` 中调用这个融合内核，而不是用 PyTorch 计算。针对本题的提示如下：

- 调试时，建议把每一步 Triton 操作的结果与 (a) 中按 tile 编写的 PyTorch 实现比较。
- `launch grid` 应设为 `(T_q, batch_size)`。因此每个 Triton 程序实例只加载一个 batch index 中的元素，也只读写 Q、O 和 L 中的一个 query tile。
- 内核中只应有一个循环，用来遍历 key tile: $1 \leq j \leq T_k$ 。
- 在循环结尾推进 block pointer。
- 使用下面的函数声明。根据给出的 block pointer，应能推断其余 pointer 的设置方式：

```
@triton.jit
def flash_fwd_kernel(
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq,
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer by batch_index times its batch stride.
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )
    ...
```

其中 `scale = 1 / sqrt(d)`，而 `Q_TILE_SIZE` 与 `K_TILE_SIZE` 分别是 `B_q` 和 `B_k`；稍后可以继续调优。

以下补充建议有助于避免精度问题：

- 片上 buffer (`O_i`、`l`、`m`) 的数据类型应为 `tl.float32`。如果向输出 buffer 累加，使用 `acc` 参数，例如 `acc = tl.dot(..., acc=acc)`。
- 先把 `P_tilde_i^(j)` 转换为 `V^(j)` 的数据类型再做乘法；把 `O_i` 写入全局显存前，也要转成适当类型。用 `tensor.to` 转换；用 `tensor.dtype` 获取 tensor 类型；用 `*_block_ptr.type.element_ty` 获取 block pointer 或 pointer 的元素类型。

提交内容： 一个 `torch.autograd.Function` 子类，使用 Triton 内核实现 FlashAttention-2 前向传播。实现 `[adapters.get_flash_autograd_function_triton]`，再运行：

```
uv run pytest -k test_flash_forward_pass_triton
```

并确认测试通过。

(c) 在 `autograd.Function` 实现的最后增加一个用于因果 mask 的标志。它是布尔值；设为 `True` 时，启用索引比较并形成因果 mask。Triton 内核中应增加对应参数：

```
is_causal: tl.constexpr
```

这是必需的类型注解。在 Triton 中构造 query 与 key 的索引向量，再比较它们以形成大小为 $B_q \times B_k$ 的方形 mask。对被 mask 的元素，在对应注意力分数矩阵 $S_i(j)$ 上加常数 $-1e6$ 。还要通过 `ctx.is_causal = is_causal` 保存该标志，供反向传播使用。

提交内容： 为使用 Triton 内核的 FlashAttention-2 `torch.autograd.Function` 前向传播增加因果 mask 标志。该参数必须可选且默认值为 `False`，保证之前的测试仍然通过。

通过重计算实现反向传播

与公式 (7)-(11) 的标准反向传播不同，公式 (13)-(19) 使用重计算后，反向传播不再需要 softmax。因此可以用一个直接的内核计算，无需在线算法。本题中，`backward` 可以通过 `torch.compile` 编译普通 PyTorch 函数来实现，不要求 Triton。

题目 flash_backward: FlashAttention-2 反向传播 (5 分)

使用 PyTorch (不是 Triton) 和 `torch.compile`，实现 FlashAttention-2 `autograd.Function` 的反向传播。实现应接收张量 Q, K, V, O, dO, L ，并返回 dQ, dK, dV 。记得计算并使用向量 D 。可以按照公式 (13)-(19) 的计算过程完成。

提交内容： 运行下面的命令测试实现：

```
uv run pytest -k test_flash_backward
```

接下来，把这个部分使用 Triton 的 FlashAttention-2 实现与普通 PyTorch Attention 的性能进行比较。

题目 flash_benchmarking: FlashAttention-2 基准测试 (5 分)

(a) 使用 `triton.testing.do_bench` 编写基准测试脚本，对比你的 FlashAttention-2 实现与普通 PyTorch 实现 (不使用 FlashAttention) 的前向和反向传播性能。

具体来说，需要报告一个表格，对 Triton 与 PyTorch 两种实现分别列出：前向延迟、反向延迟，以及端到端前向加反向延迟。开始计时前先随机生成全部所需输入，并在单块 B200 上运行。

统一使用 batch size 1 和因果 mask。遍历以下配置的笛卡尔积：

- sequence length: 从 128 到 65536 的若干个 2 的幂；
- embedding dimension: 从 16 到 128 的若干个 2 的幂；
- precision: `torch.bfloat16` 与 `torch.float32`。

根据输入大小，你可能需要调整 tile size。

提交内容： 一张结果表，在上述配置下比较 FlashAttention-2 与 PyTorch 实现的前向、反向和端到端延迟。

4.2.3 可选: Triton 反向传播

如果希望进一步练习 Triton，或希望排行榜提交更快，可以根据下面给出的分块 FlashAttention-2 反向传播算法，在 Triton 中实现反向内核。

算法 2 的关键技巧是把 P 计算两次：一次用于 dQ ，另一次用于 dK 与 dV 。这样可避免线程块之间的同步，也就不必使用速度较慢的原子操作。

算法 2: 分块 FlashAttention-2 反向传播

```

 $Q, O, d_0 \in \mathbb{R}^{(N_q \times d)}$ ,  $K, V \in \mathbb{R}^{(N_k \times d)}$ ,  $L \in \mathbb{R}^{(N_q)}$ 
tile  $q \times B_q \times B_k$ 

1.  $D = \text{rowsum}(O \circ d_0) \in \mathbb{R}^{(N_q)}$ 
2.  $Q \times d_0 \times d_0 \times T_q = \text{ceil}(N_q / B_q) \times \text{tile}$ 
   tile  $q \times B_q \times d$ 
3.  $K \times V \times T_k = \text{ceil}(N_k / B_k) \times \text{tile}$ 
   tile  $k \times B_k \times d$ 
4.  $L \times D \times T_q \times \text{tile} \times B_q$ 

5.  $j = 1, \dots, T_k$ 
  a.  $K^{(j)} \times V^{(j)}$ 
  b.  $dK_{\theta}^{(j)} = dV_{\theta}^{(j)} = 0 \in \mathbb{R}^{(B_k \times d)}$ 
  c.  $i = 1, \dots, T_q$ 
    i.  $Q_i \times d_{0_i}$ 
    ii.  $S_i^{(j)} = Q_i (K^{(j)})^T / \sqrt{d} \in \mathbb{R}^{(B_q \times B_k)}$ 
    iii.  $P_i^{(j)} = \exp(S_i^{(j)} - L_i) \in \mathbb{R}^{(B_q \times B_k)}$ 
    iv.  $dV_i^{(j)} = dV_{i-1}^{(j)} + (P_i^{(j)})^T d_{0_i}$ 
    v.  $dP_i^{(j)} = d_{0_i} (V^{(j)})^T$ 
    vi.  $dS_i^{(j)} = P_i^{(j)} \circ (dP_i^{(j)} - D_i)$ 
    vii.  $dK_i^{(j)} = dK_{i-1}^{(j)} + (dS_i^{(j)})^T Q_i / \sqrt{d}$ 
  d.  $dK_{T_q}^{(j)} \times dV_{T_q}^{(j)}$ 
   tile  $k \times d \times B_k \times j \times \text{tile}$ 

6.  $i = 1, \dots, T_q$ 
  a.  $Q_i \times d_{0_i}$ 
  b.  $dQ_i^{(0)} = 0 \in \mathbb{R}^{(B_q \times d)}$ 
  c.  $j = 1, \dots, T_k$ 
    i.  $K^{(j)} \times V^{(j)}$ 
    ii.  $S_i^{(j)} = Q_i (K^{(j)})^T / \sqrt{d}$ 
    iii.  $P_i^{(j)} = \exp(S_i^{(j)} - L_i)$ 
    iv.  $dP_i^{(j)} = d_{0_i} (V^{(j)})^T$ 
    v.  $dS_i^{(j)} = P_i^{(j)} \circ (dP_i^{(j)} - D_i)$ 
    vi.  $dQ_i^{(j)} = dQ_i^{(j-1)} + dS_i^{(j)} K^{(j)} / \sqrt{d}$ 
  d.  $dQ_i^{(T_k)}$ 
   tile  $q \times d \times B_q \times i \times \text{tile}$ 

7.  $dQ \times dK \times dV$ 

```

5 分布式数据并行训练

在下一部分，我们将研究如何用多块 GPU 训练语言模型，重点是数据并行。首先介绍 PyTorch 中的分布式通信，然后研究一个朴素的分布式数据并行实现，最后实现并测试多种通信效率改进方法。

5.1 PyTorch 单节点分布式通信

先看一个简单的 PyTorch 分布式应用，目标是生成四个随机整数张量并计算它们的和。

在下面的分布式情形中，程序会启动四个 worker process，每个进程各自生成一个随机整数张量。为了跨 worker 求和，调用 all-reduce 集合通信操作。它会把每个进程原来的数据张量原地替换为 all-reduce 结果，也就是总和。

代码如下：

```

import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(
        fn=distributed_demo,
        args=(world_size,),
        nprocs=world_size,

```

```
    join=True,
)
```

运行后得到类似输出。符合预期：一开始每个 worker 持有不同的数据张量；all-reduce 把所有 worker 的张量相加后，每个 worker 上的 **data** 都被原地改成同一个求和结果。

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

如果多次运行该脚本，会发现打印顺序并不确定。因为应用在分布式环境中运行，无法控制各命令执行的精确先后顺序；唯一保证是 all-reduce 完成后，各进程持有逐 bit 完全一致的结果张量。

再仔细看脚本：`mp.spawn` 会启动 `nprocs` 个进程，并让它们使用给定 `args` 执行 `fn`。实际调用形式是 `fn(rank, *args)`，其中 `rank` 是 worker process 的索引，取值范围为 0 到 `nprocs-1`。因此，`distributed_demo` 必须把整数 `rank` 作为第一个位置参数。此外还传入 `world_size`，它表示 worker process 总数。

每个 worker 都属于一个 process group，该组通过 `dist.init_process_group` 初始化。process group 表示一组会通过共同 master 相互协调通信的 worker process。master 由 IP 地址与端口定义，并且由 rank 0 的进程担任。all-reduce 之类的集合通信会在 process group 的所有进程上执行。

本例使用 "gloo" backend 初始化 process group，但也有其他 backend。尤其是，"nccl" 会使用 NVIDIA NCCL 集合通信库，对 CUDA 张量通常性能更好。不过 NCCL 只能在有 GPU 的机器上使用，Gloo 则可在纯 CPU 机器上运行。分布式 GPU 训练应始终使用 NCCL；只有在没有 GPU、需要本地开发调试时才用 Gloo。本例采用 Gloo，正是为了能在纯 CPU 环境中执行。

运行多 GPU 任务时，要确保不同 rank 使用不同 GPU。一种方法是在 `setup` 中调用 `torch.cuda.set_device(rank)`；之后 `tensor.to("cuda")` 会自动把张量移动到指定设备。另一种方法是为每个 rank 显式创建设备字符串，例如 `device = f"cuda:{rank}"`，并在每次数据移动时把它作为目标，例如 `tensor.to(f"cuda:{rank}")`。

术语

在本作业其余部分以及网上的相关资料中，你会遇到以下 PyTorch 分布式通信术语。虽然本作业主要处理单节点、多进程的分布式训练，但这些术语有助于理解一般的分布式训练。参见图 3。

node (节点)：网络中的一台机器。

worker (工作进程/工作实例)：参与分布式训练的一个程序实例。本作业中每个 worker 只有一个进程，因此会交替使用 worker、process 与 worker process。不过在实践中，一个 worker 可能会使用多个进程，例如加载训练数据，因此这些概念并不总是等价。

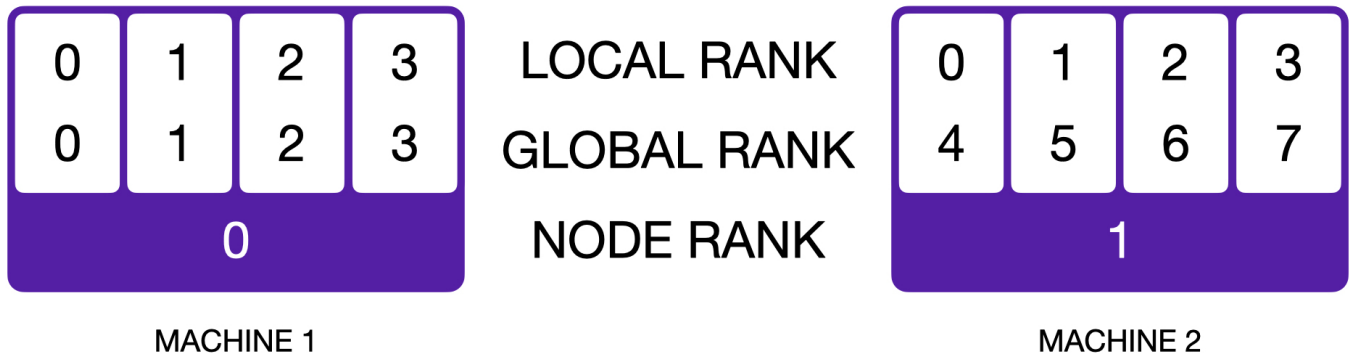
world size (全局规模)：一个 process group 中 worker 的总数。

global rank (全局编号)：在 0 到 `world_size-1` 范围内，唯一标识 process group 中某个 worker 的整数 ID。例如 world size 为 2 时，一个进程的 global rank 为 0 (master process)，另一个为 1。

local world size (本地规模)：应用跨多个节点运行时，某个节点本地运行的 worker 数。例如，在 2 个节点上各启动 4 个 worker，则 world size 为 8，而 local world size 为 4。单节点运行时，一个 worker 的 local world size 与全局 world size 相同。

local rank (本地编号)：在 0 到 `local_world_size-1` 范围内，唯一标识一台机器上本地 worker 索引的整数 ID。例如，在 2 个节点上各启动 4 个进程，则每个节点都有 local rank 为 0、1、2、3 的 worker。单节点多进程应用中，进程的 local rank 等于 global rank。

图 3：一个在 2 个节点上运行的分布式应用，world size 为 8。每个 worker process 同时由 global rank (0-7) 和 local rank (0-3) 标识。图片来源：https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html。



双节点分布式应用中的 local rank 与 global rank

5.1.1 分布式应用基准测试的最佳实践

本部分会对分布式应用做基准测试，以更好地理解通信开销。建议遵循以下做法：

- 在可能的情况下，把对比实验放在同一台机器上运行，以控制变量。
- 正式计时前先执行若干预热步骤。NCCL 通信调用尤其需要预热；通常 5 次迭代已经足够。
- 在 GPU 上做基准测试时，调用 `torch.cuda.synchronize()` 等待 CUDA 操作完成。即使通信操作使用 `async_op=False`，也必须这样做；该选项只保证操作已经排入 GPU 队列，并不代表通信已经结束。
- 如果使用多个 CUDA stream，在高级情形下可能还需要跨 stream 显式同步，保证输出已准备好供之后的操作使用。详见：<https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams>。
- 不同 rank 上的计时可能略有差异，通常会汇总各 rank 的测量值以提高估计质量。可以用 all-gather 集合通信，特别是 `dist.all_gather_object`，收集所有 rank 的结果。
- 一般来说，先在本地 CPU 上用 Gloo 调试，再按各题要求在 GPU 上用 NCCL 测试。切换 backend 应只需修改 `init_process_group` 调用和张量设备转换。

关于 `async_op=False` 的更多细节见：<https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386>。

题目 distributed_communication_single_node: 分布式通信（单节点）（5 分）

编写脚本，在单节点、多进程环境中测量 all-reduce 操作的运行时间。上面的示例代码可以作为起点。改变以下设置进行实验：

- all-reduce 数据量：** float32 张量，大小分别为 1 MB、10 MB、100 MB、1 GB；
- GPU/进程数：** 2、4 或 6。

资源要求： 最多使用 6 块 GPU；每次基准测试应在 5 分钟内完成。

提交内容： 用图和/或表比较不同设置，并用 2-3 句话讨论结果以及各因素之间的相互作用。

5.2 朴素的分布式数据并行实现

理解了 PyTorch 分布式应用的基础后，下面构建一个最小化的分布式数据并行（DDP）训练实现。

数据并行把一个 batch 拆到多台设备（例如 GPU）上，从而可以用单台设备容纳不了的大 batch 训练。例如，4 台设备各自最多处理 batch size 32，通过数据并行就能获得 128 的有效 batch size。

朴素 DDP 训练步骤如下。开始时，每台设备都构造一个随机初始化的模型。使用 broadcast 集合通信，把 rank 0 的模型参数发送给其他所有 rank。训练开始时，每台设备都有完全相同的模型参数和优化器状态，例如 Adam 中累积的梯度统计量。

1. 对含 n 个样本的 batch 做分片，每台设备接收互不重叠的 n/d 个样本，其中 d 是数据并行设备数。 d 应整除 n ；否则某些 rank 工作更多，整个步骤会受最慢 rank 限制。
2. 每台设备使用自己的本地模型参数，在这 n/d 个样本上做前向传播和反向传播，得到局部梯度。此时每台设备持有的梯度只来自自己收到的样本。
3. 使用 all-reduce 集合通信，对不同设备的梯度取平均。完成后，每台设备都持有基于全部 n 个样本的平均梯度。
4. 每台设备执行优化器更新，更新本地参数副本。从优化器角度看，它只是在优化一个本地模型。因为所有设备从相同的模型参数和优化器状态开始，并且每次迭代使用相同的平均梯度，所以各设备的参数和优化器状态会始终保持同步。至此完成一次训练迭代，可以重复上述过程。

题目 naive_ddp: 朴素 DDP (5 分)

提交内容： 实现一种朴素的分布式数据并行训练：在反向传播完成后，逐个对参数梯度执行 all-reduce。为了测试，实现 [adapters.get_ddp]，并可选实现 [adapters.ddp_on_after_backward]，然后运行：

```
uv run pytest tests/test_ddp.py
```

题目 naive_ddp_benchmarking: 朴素 DDP 基准测试 (3 分)

在这个朴素 DDP 实现中，每次反向传播后，各 rank 会逐个对参数梯度执行 all-reduce。为了理解数据并行训练的开销，请编写脚本，使用这个朴素 DDP 实现对先前编写的语言模型做基准测试。

测量每个训练步骤的总时间，以及其中用于梯度通信的时间比例。使用单节点环境：1 node × 2 GPUs，模型采用 2.1.2 节的 xl 规模。

提交内容： 描述基准测试设置，并报告每种设置下每次训练迭代的总时间与梯度通信时间。

5.3 改进最小 DDP 实现

5.2 节的最小 DDP 实现有两个关键局限：

1. 每个参数张量都会单独执行一次 all-reduce。每次通信调用都有固定开销，因此把多个通信合并成批次，可能有助于降低总开销。
2. 它要等反向传播全部结束后才开始传输梯度。但反向传播的梯度是逐步计算出来的：某个参数的梯度一旦准备好，就可立即通信，不必等待其他参数。这样可以让梯度通信与反向计算重叠，降低数据并行训练的额外开销。

本部分将依次解决这两个问题，并测量它们对训练速度的影响。

5.3.1 减少通信调用次数

与其为每个参数张量发起一次通信，不如把要 all-reduce 的梯度连接为一个张量，再跨全部 rank 对这个合并张量执行一次 all-reduce。可能会用到：

```
torch._utils._flatten_dense_tensors
torch._utils._unflatten_dense_tensors
```

题目 minimal_ddp_flat_benchmarking: 扁平梯度最小 DDP 基准测试 (2 分)

修改最小 DDP 实现，把所有参数的梯度展平并合并成一个张量后通信。使用此前相同条件--1 node × 2 GPUs、2.1.2 节的 xl 模型--把它与“为每个参数张量单独发起 all-reduce”的最小 DDP 实现进行性能比较。

提交内容： 报告只用一次批量 all-reduce 的 DDP 每次训练迭代耗时，以及用于梯度通信的时间；再用 1-2 句话对比批量通信和逐参数通信。

5.3.2 让计算与单个参数梯度通信重叠

批量合并通信调用可以降低大量小型 all-reduce 的发起开销，但全部通信时间仍直接位于关键路径上。为进一步降低开销，可以利用一个事实：反向传播从损失向输入方向逐层计算梯度，因此每个参数的梯度一旦就绪，就可立即 all-reduce。这样能让反向传播计算与梯度通信重叠。

先实现并测试一个 DDP 包装器：反向传播期间，每个参数张量的梯度一旦准备好，就异步执行 all-reduce。下面的提示可能有用。

反向钩子。 如果要在某个参数的梯度完成累加后自动调用函数，可使用 `register_post_accumulate_grad_hook`。用法见：https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html。

异步通信。 PyTorch 的全部集合通信操作都支持同步执行 (`async_op=False`) 与异步执行 (`async_op=True`)。

同步调用会阻塞，直到集合通信已排入 GPU 队列。由于 CUDA 操作本身是异步的，这不代表 GPU 操作已经完成；不过，之后依赖该输出的函数调用会按正确顺序执行。

异步调用会立即返回一个 distributed request handle。函数返回时，集合通信甚至不保证已经排入 GPU 队列，更不用说完成。对返回的通信 handle 调用 `handle.wait()`，可以等到该操作进入 GPU 队列，此后输出即可供后续操作使用。

下面分别用同步与异步方式，对张量列表中的每个张量执行 all-reduce：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous: block until each operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous: return immediately, then wait at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
# Possibly run commands that do not depend on the all_reduce results.
# ...

# Ensure all-reduce calls have been queued, so dependent operations may queue.
for handle in handles:
    handle.wait()
handles.clear()
```

题目 `ddp_overlap_individual_parameters`：通信与单个参数计算重叠的 DDP（5 分）

实现一个 Python 类来处理分布式数据并行训练。该类应包装任意 PyTorch `nn.Module`，负责在训练前广播权重，使所有 rank 具有相同初始参数，并负责发出用于梯度平均的通信调用。

建议采用以下公开接口：

```
def __init__(self, module: torch.nn.Module):
```

接收一个已经实例化、需要并行化的 PyTorch `nn.Module`，构造负责跨 rank 同步梯度的 DDP 容器。

```
def forward(self, *inputs, **kwargs):
```

用给定的位置参数和关键字参数调用被包装模块的 `forward()`。

```
def finish_gradient_synchronization(self):
```

被调用时，等待异步通信调用在 GPU 上完成必要的排队与同步。

使用这个类进行分布式训练时，把模块交给它包装，并在执行 `optimizer.step()` 前调用 `finish_gradient_synchronization()`，保证依赖梯度的优化器更新可以安全排入队列：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
```

```
logits = ddp_model(x)
loss = loss_fn(logits, y)
loss.backward()
ddp_model.finish_gradient_synchronization()
optimizer.step()
```

提交内容：实现一个 DDP 容器类，使梯度通信与反向传播计算重叠。为了测试，先实现 `adapter [adapters.get_ddp]`；根据实现方式，你可能还需实现可选的 `[adapters.ddp_on_after_backward]`。然后运行：

```
uv run pytest tests/test_ddp.py
```

建议多次运行测试（例如 5 次），确认它能稳定通过。

题目 `ddp_overlap_individual_parameters_benchmarking`：单参数通信重叠 DDP 基准测试（1 分）

(a) 测量你的 DDP 实现性能，其中反向传播计算与各参数梯度的通信相互重叠。使用相同设置--1 个节点、2 块 GPU、2.1.2 节的 `xl` 模型--把它与之前两种设置比较：逐参数 all-reduce 的最小 DDP，以及把所有参数连接后只做一次 all-reduce 的 DDP。

提交内容：报告通信与反向传播重叠时，每次训练迭代的实测时间，并用 1-2 句话比较结果。

(b) 对基准测试代码使用 Nsight profiler，在 `1 node × 2 GPUs`、`xl` 模型设置下，对比初始 DDP 与通信重叠实现。直观比较两条轨迹，并提供性能分析器截图，展示一个实现中计算与通信有重叠，而另一个没有。

提交内容：2 张截图：一张来自初始 DDP，另一张来自让计算与通信重叠的 DDP；图中应能直观看到通信是否与反向传播重叠。

6 优化器状态分片

分布式数据并行概念简单，而且通常非常有效，但要求每个 rank 都保存一份完整、独立的模型参数与优化器状态副本。这种冗余会带来显著的显存成本。例如，AdamW 对每个参数维护两个浮点数，因此优化器状态占用的显存是模型权重的两倍。

S. Rajbhandari 等 [5] 提出了多种降低数据并行冗余的方法：把（1）优化器状态、（2）梯度和（3）参数分别跨 rank 划分，并在需要时在 worker 之间通信。

本部分将实现一个简化的优化器状态分片版本，以降低每个 rank 的显存用量。每个 rank 的优化器实例不再维护所有参数的优化器状态，而只负责大约 `1 / world_size` 的参数子集。每个 rank 执行优化器更新时，只更新属于自己分片的模型参数；然后把更新后的参数广播给其他 rank，保证每次优化器更新后各 rank 的模型参数仍保持同步。

题目 `optimizer_state_sharding`：优化器状态分片（15 分）

实现一个用于优化器状态分片的 Python 类。该类应包装任意 PyTorch `optim.Optimizer`，并在每次优化器更新后负责同步已更新的参数。建议采用以下公开接口：

```
def __init__(
    self,
    params,
    optimizer_cls: Type[Optimizer],
    **kwargs: Any,
):
```

初始化分片状态优化器。`params` 是需要优化的参数集合；如果用户希望模型不同部分使用不同超参数，例如不同学习率，它也可以是参数组。所有这些参数会跨 rank 分片。`optimizer_cls` 指定被包装的优化器类型，例如 `optim.AdamW`。其余关键字参数转发给 `optimizer_cls` 的构造函数。务必在该方法中调用 `torch.optim.Optimizer` 父类构造函数。

```
def step(self, closure, **kwargs):
```

使用给定的 `closure` 和关键字参数，调用被包装优化器的 `step()`；更新参数后，与其他 rank 同步。

```
def add_param_group(self, param_group: dict[str, Any]):
```

向分片优化器添加参数组。父类构造函数在初始化期间会调用此方法，训练期间也可能调用，例如逐步解冻模型层。因此，该方法需要正确地把模型参数分配给各个 rank。

提交内容： 实现一个处理优化器状态分片的容器类。为了测试，先实现 `adapter [adapters.get_sharded_optimizer]`，再运行：

```
uv run pytest tests/test_sharded_optimizer.py
```

建议多次运行测试（例如 5 次），确认能够稳定通过。

实现优化器状态分片后，下面分析它对训练峰值显存和运行时间开销的影响。

题目 `optimizer_state_sharding_accounting`：优化器状态分片核算（5 分）

(a) 编写脚本，分析语言模型在启用和不启用优化器状态分片时的峰值显存。使用标准配置：1 个节点、2 块 GPU、xl 模型。分别报告模型初始化完成后、优化器更新紧邻之前、优化器更新紧邻之后的峰值显存。结果是否符合预期？拆解每种设置的显存构成，例如参数占多少、优化器状态占多少等。

提交内容： 2-3 句话，报告峰值显存结果及模型和优化器不同组件的显存构成。

(b) 这里的优化器状态分片实现如何影响训练速度？对标准配置--1 个节点、2 块 GPU、xl 模型--分别测量启用和不启用分片时的每次迭代耗时。

提交内容： 2-3 句话，报告计时结果。

(c) 本作业的优化器状态分片方法与 ZeRO Stage 1 有何不同？ZeRO Stage 1 在 S. Rajbhandari、J. Rasley、O. Ruwase 与 Y. He 的论文 [5] 中记作 ZeRO-DP P_{os}。

提交内容： 2-3 句话总结差异，尤其关注显存和通信量。

7 完全分片数据并行

优化器状态分片结合数据并行后，可以沿数据并行轴拆分优化器状态与激活。然而，模型权重仍然是重复的：每块 GPU 都保存完整副本。

可以把数据并行（DP）轴进一步变成完全分片数据并行（FSDP）轴来解决问题。在 FSDP 中，每块 GPU 只保存每个权重张量的一段分片；但为了执行前向或反向传播，需要通过 all-gather 从其他 GPU 拉取分片并拼出完整权重。

为了不让 GPU 计算等待通信完成，多数 FSDP 实现都会在真正使用某层之前，提前调度该层的 all-gather。这样在需要权重时，它们已准备完毕，通信就不会阻塞计算。如果通信能跟上计算并且调度足够及时，权重分片通信将不在关键路径上，因而几乎没有额外时间成本。

有些层的显存和计算量都很小，此时传输的延迟开销得不偿失。这些层不应由 FSDP 分片。在本作业架构中，主要是各种 norm 层；需要分片的则是 embedding 层与每一个 linear 层。

主权重必须保留为 FP32，因为反复进行累加的数值对精度敏感；但实际用于计算的权重不必是 FP32。混合精度下，使用权重前总会转换成低精度计算类型，因此可以在通信前就转换为低精度，以节省带宽。

题目 `fsdp`：完全分片数据并行（15 分）

实现一个用于 FSDP 训练的 Python 类。该类应包装任意 PyTorch `nn.Module`（完整模型），并通过 hook 或 wrapper 接管其中全部 Linear 与 Embedding 层。

建议采用以下公开接口：

```
def __init__(
    self,
    module: torch.nn.Module,
    compute_dtype: torch.dtype | None = None,
):
```

接收一个已经实例化、需要并行化的 PyTorch `nn.Module`，构建一个 FSDP 模块，负责权重 all-gather 与梯度 reduce-scatter。

你的 hook 或模块 wrapper 必须及时 all-gather 权重，使其可用于前向传播。为了限制显存，只能在“当前层之前的第二层”完成前向传播后，才开始 gather 当前层权重。反向传播时，也需要通过 hook 或 wrapper 执行 all-gather，让权重可用于计算。梯度就绪后，应把它们 reduce-scatter 到相应 rank。使用后务必释放已经 gather 的权重。

如果提供 `compute_dtype`，应在通信或计算前把权重转换成该类型；但主权重和优化器更新仍必须保持 FP32。

```
def forward(self, *inputs, **kwargs):
```

使用给定位置参数和关键字参数，调用被包装模块的 `forward()`。

```
def finish_gradient_synchronization(self):
```

被调用时，等待 GPU 上的异步通信完成必要同步。

提交内容：实现完全分片数据并行训练容器。该容器的每个分片都应兼容作业 1 中的标准 AdamW。实现 `adapter [adapters.get_fsdp]`，再运行：

```
uv run pytest tests/test_fsdp.py
```

建议多次运行测试（例如 5 次），以发现潜在 race condition。

题目 fsdp_accounting: FSDP 核算 (5 分)

(a) 根据第 6 节的分析，实现 FSDP 后预计能从峰值显存中节省多少？计算时可以忽略 all-gather 权重所需的预分配 buffer。

提交内容：2-3 句话报告结论。

(b) 在两块 GPU 上分析 xl 模型，重点观察权重 all-gather。通信是否能在前向传播需要这些权重之前完成？

提交内容：2-3 句话报告计时，并附 Nsight 截图支持结论。

8 分析并行策略

训练还可以沿更多轴做并行。常见策略包括：

- **数据并行 (Data Parallelism, DP)**：把 batch 拆到多台设备，每台设备为自己的数据分片计算梯度，然后跨设备对梯度取平均。
- **完全分片数据并行 (Fully-Sharded Data Parallelism, FSDP)**：在数据并行基础上，进一步跨设备拆分优化器状态、梯度和权重，以降低显存用量。前向和反向传播时，设备需要从其他设备收集权重分片。
- **张量并行 (Tensor Parallelism, TP)**：沿输入维或输出维对权重矩阵分片。每台设备只计算与自身权重分片对应的激活，之后跨设备归约或收集激活。
- **流水线并行 (Pipeline Parallelism, PP)**：沿层把模型划分为多个 stage，每个 stage 在不同设备上运行。
- **专家并行 (Expert Parallelism, EP)**：把混合专家模型中的不同 expert 分配到不同设备，每台设备计算自己负责的 expert 输出。

本节会在简化场景下做一些基础计算，用来选择并组合并行策略。首先关注 DP、FSDP、TP 及其组合。基本方法是计算每种策略的通信成本并与计算成本比较，从而判断在通信成为瓶颈之前最多能扩展到多少设备。

若想更详细地了解 TPU/GPU 拓扑与并行策略，推荐阅读 TPU Scaling Book (J. Austin 等 [6])。若想深入了解流水线并行，可读 The Ultra-Scale Playbook 的附录 (Nouamane Tazi Ferdinand Mom 等 [7])。两本资料的其他部分也包含很多有用信息。

8.1 集合通信原语

第一步是理解通信原语。在简化模型中，假设有 N 台设备，编号为 $0, \dots, N-1$ ，任意两台设备之间都有连接。再假设每台设备都有 W 的 egress（出站）带宽，即每台设备可以每秒向另一台设备发送 W 字节。下面考虑如何实现 gather 与 reduce。

Ring all-gather

ring all-gather 是 all-gather 的常见实现。开始时，每台设备 i 持有一个大小为 S/N 的分块 x_i ；结束时，每台设备都持有完整的 $x = [x_0, \dots, x_{(N-1)}]$ ，总大小为 S 字节。

把所有设备排成一个环。每一步中，每台设备把当前分块发送给右侧下一台设备，并保存从左侧设备收到的分块。随后重复此过程：每台设备继续向右发送刚收到的分块，同时从左边接收一个新分块。经过 $N-1$ 步，每台设备都获得完整张量。

在理想化设置中，每一步每台设备会同时发送大小为 S/N 的分块，出站带宽为 W ，共执行 $N-1$ 步。因此 ring all-gather 耗时：

$$T_{\text{ring_all_gather}} = ((N-1) / N) \cdot (S / W)$$

Ring reduce-scatter

在 reduce-scatter 中，每台设备 i 最初持有一个大小为 S 的完整张量 $x^{(i)}$ 。目标是计算：

$$y = \sum_{i=0}^{(N-1)} x^{(i)}$$

但最终每台设备 i 只保留 y 中大小为 S/N 的一个分块 y_i 。

同样先把设备排列成环。每台设备把自己的 $x^{(i)}$ 划分为 N 个大小为 S/N 的分块： $[x_0^{(i)}, \dots, x_{(N-1)}^{(i)}]$ 。然后像 ring all-gather 一样传递分块，但每台设备在继续传递前，会把自己的贡献加到该分块所存的局部和中。

具体而言，对步骤 $t = 1, \dots, N-1$ ，设备 i 执行：

- 若 $t = 1$ ，初始化 $y \leftarrow x^{(i)}$ ，保存当前局部和；
- 把分块 $y_{((i-t) \bmod N)}$ 发送给设备 $(i+1) \bmod N$ ；
- 从设备 $(i-1) \bmod N$ 接收分块 $z_{((i-t-1) \bmod N)}$ ；
- 更新局部和：

$$y_{((i-t-1) \bmod N)} \leftarrow y_{((i-t-1) \bmod N)} + z_{((i-t-1) \bmod N)}$$

经过 $N-1$ 步，设备 i 获得分块 y_i 的完整求和结果。因此 ring reduce-scatter 与 ring all-gather 耗时相同：

$$T_{\text{ring_reduce_scatter}} = ((N-1) / N) \cdot (S / W)$$

Ring all-reduce

最后实现 ring all-reduce。开始时，每台设备 i 持有大小为 S 的完整张量 $x^{(i)}$ ；结束时，每台设备都获得归约结果：

$$y = \sum_{i=0}^{(N-1)} x^{(i)}$$

把 all-reduce 实现为一次 ring reduce-scatter，再接一次 ring all-gather，因此总耗时为：

$$T_{\text{ring_all_reduce}} = 2 \cdot ((N-1) / N) \cdot (S / W)$$

题目 alternate_ring_all_reduce: 另一种 ring all-reduce (1分)

不要把 all-reduce 实现为 ring reduce-scatter 加 ring all-gather，而改用下面的算法。

对步骤 $t = 1, \dots, N-1$ ，设备 i 执行：

- 若 $t = 1$ ，初始化 $y \leftarrow x^{(i)}$ ，保存当前局部和；
- 把 $x^{((i-t+1) \bmod N)}$ 发送给设备 $(i+1) \bmod N$ ；
- 从设备 $(i-1) \bmod N$ 接收 $x^{((i-t) \bmod N)}$ ；
- 更新自己的局部和副本：

$$y \leftarrow y + x^{((i-t) \bmod N)}$$

在与上面相同的设置中--每台设备的出站带宽为 W ，每个 $x^{(i)}$ 的大小为 S --该算法需要多久？

提交内容： 用 S 、 N 和 W 表示答案，并用一句话说明理由。

8.2 分析数据并行

有了通信原语，下面开始分析并行策略。我们只分析单个 FFN 层的并行化。给定输入 x ，其前向传播为：

```
x1 = x W1           (20)
x2 = x W2           (21)
z  = f(x1) * x2      (22)
y  = z W3           (23)
```

其中， x 的形状为 (B, D) ； $W1$ 与 $W2$ 的形状为 (D, D_{FF}) ； $W3$ 的形状为 (D_{FF}, D) ； f 是逐元素激活函数，例如 SiLU； $*$ 表示逐元素乘法。

把反向传播显式写出也很有帮助。给定形状为 (B, D) 的 dy ，反向传播为：

```
dz  = dy W3^T       (24)
dx2 = dz * f'(x1)   (25)
dx1 = dz * f'(x1) * x2 (26)
dx  = dx1 W1^T + dx2 W2^T (27)
dW3 = z^T dy        (28)
dW2 = x^T dx2       (29)
dW1 = x^T dx1       (30)
```

其中 $*$ 仍表示逐元素乘法。

在使用 N_{DP} 台设备的数据并行中，输入 x 沿 batch 维被拆成分片 $x^{(i)}$ ，每个分片大小为 $(B/N_{DP}, D)$ 。前向传播不需要任何集合通信，直接得到形状为 $(B/N_{DP}, D)$ 的激活 $y^{(i)}$ 。

反向传播照常在 batch 分片激活上执行。设备 i 最终得到：

```
dW3^(i) = (z^(i))^T dy^(i) (31)
dW2^(i) = (x^(i))^T dx2^(i) (32)
dW1^(i) = (x^(i))^T dx1^(i) (33)
```

这些结果只对本设备输入分片中的 B/N_{DP} 个外积求了局部和，而不是全部 B 个外积。因此还需要跨设备执行 all-reduce，得到完整梯度 $dW3$ 、 $dW2$ 和 $dW1$ 。

题目 data_parallel_calcs：数据并行计算（3 分）

现在已经具备计算“数据并行何时受通信瓶颈限制”所需的全部条件。令设备加速器速度为 C FLOP/s，每台设备的出站带宽为 W bytes/s。由此可以计算计算时间和通信时间。因为两者可以重叠，所以当通信时间大于计算时间时，系统就受通信瓶颈限制。假设所有权重和激活都是 FP16，即每个元素 2 字节。

(a) 使用 N_{DP} 路数据并行时，计算一次反向传播需要多少 FLOPs？忽略所有非矩阵乘法操作。回忆：矩阵乘法 $(A, B)(B, C) \rightarrow (A, C)$ 需要 $2ABC$ FLOPs。

提交内容： 用 B 、 D 、 D_{FF} 和 N_{DP} 表示答案，并用一句话说明理由。

(b) 使用 N_{DP} 路数据并行时，反向传播需要多少通信时间？

提交内容： 用 B 、 D 、 D_{FF} 、 N_{DP} 、 W 中必要的变量表示答案，并用一句话说明理由。

(c) 固定其他参数后，在通信成为瓶颈之前， N_{DP} 最大可以是多少？

提交内容： 一个不等式，一边是 N_{DP} ，另一边由 B 、 D 、 D_{FF} 、 C 、 W 中必要的变量构成；再用一句话说明理由。

8.3 分析完全分片数据并行

下面分析 FSDP。和 DP 一样，FSDP 会对输入与激活的 batch 维进行分片。此外，为了节省显存，它还会分片优化器状态、梯度和权重。

权重可以沿任一维度分片，于是设备 i 上会得到 $W1^{(i)}$ 、 $W2^{(i)}$ 、 $W3^{(i)}$ ；每个分片大小均为：

$$D \cdot D_{FF} / N_{FSDP}$$

在前向传播中，仍对 batch 分片输入执行数据并行前向传播。但开始前，需要先跨设备 all-gather 权重分片：

```

W1 = all-gather({W1^(i)}_(i=0)^(N_FSDP-1))          (34)
W2 = all-gather({W2^(i)}_(i=0)^(N_FSDP-1))          (35)
W3 = all-gather({W3^(i)}_(i=0)^(N_FSDP-1))          (36)

□□ batch-sharded forward pass                        (37)

```

all-gather 的具体时间点可以灵活安排：只需保证某个权重被使用前已经完成 gather，并在使用后立刻丢弃，以降低显存成本。为简化推导，本节把三次 all-gather 连续写在一起。

反向传播同样需要先 all-gather 权重分片。随后可以执行数据并行反向传播，但不再需要 all-reduce，因为每台设备只需获得自己那一段权重分片的梯度。因此改为 reduce-scatter：

```

W1 = all-gather({W1^(i)}_(i=0)^(N_FSDP-1))          (38)
W2 = all-gather({W2^(i)}_(i=0)^(N_FSDP-1))          (39)
W3 = all-gather({W3^(i)}_(i=0)^(N_FSDP-1))          (40)

□□ batch-sharded backward pass                      (41)

dW1^(i) = reduce-scatter({dW1^(i)}_(i=0)^(N_FSDP-1)) (42)
dW2^(i) = reduce-scatter({dW2^(i)}_(i=0)^(N_FSDP-1)) (43)
dW3^(i) = reduce-scatter({dW3^(i)}_(i=0)^(N_FSDP-1)) (44)

```

注意，上标 (i) 的分片记号在此被复用了：reduce-scatter 的输入，是对完整权重计算出的局部梯度和；输出则是对应权重分片的完整梯度和。

题目 fsdp_calcs：完全分片数据并行计算（3 分）

在与数据并行计算相同的设置下，分析 FSDP 何时成为通信瓶颈。

(a) 使用 N_{FSDP} 路 FSDP 时，反向传播需要多少 FLOPs？前向传播呢？

提交内容： 两个用 B 、 D 、 D_{FF} 、 N_{FSDP} 表示的答案，并各用一句话说明理由。

(b) 使用 N_{FSDP} 路 FSDP 时，反向传播需要多少通信时间？前向传播呢？

提交内容： 两个答案，分别用 B 、 D 、 D_{FF} 、 N_{FSDP} 、 W 中必要的变量表示，并各用一句话说明理由。

(c) 固定其他参数后，在反向传播受到通信瓶颈限制之前， N_{FSDP} 最大可以是多少？前向传播呢？

提交内容： 两个不等式，一边是 N_{FSDP} ，另一边由 B 、 D 、 D_{FF} 、 C 、 W 中必要的变量构成；并各用一句话说明理由。

8.4 分析张量并行

实践中，FSDP 经常与张量并行（Tensor Parallelism, TP）结合。在 TP 中，沿每个权重矩阵的输入维或输出维，把矩阵分片到多台设备。沿输入维分片通常称为 row parallel（行并行），沿输出维分片通常称为 column parallel（列并行）。

假设要对形状为 (B, D) 的 x 与形状为 (D, D_{FF}) 的 W 计算 xW 。

在列并行中，每个权重分片 $W^{(i)}$ 的形状为 $(D, D_{\text{FF}}/N_{\text{TP}})$ ，有：

```

xW = all-gather({x W^(i)}_(i=0)^(N_TP-1))          (45)

```

在行并行中，每个 $W^{(i)}$ 的形状为 $(D/N_{\text{TP}}, D_{\text{FF}})$ 。做矩阵乘法前，每台设备还要把输入 x 缩窄为形状 $(B, D/N_{\text{TP}})$ 的切片 $x^{(i)}$ 。于是：

```

xW = all-reduce({x^(i) W^(i)}_(i=0)^(N_TP-1))      (46)

```

为了并行化 FFN，本作业采用一种特定 TP 配置： $W1$ 和 $W2$ 使用列并行，即沿输出维分片； $W3$ 使用行并行，即沿输入维分片。由于行并行权重只需要输入的一段切片，这种布局允许我们跳过列并行权重之后本来需要的 all-gather。

给定形状为 (B, D) 的输入 x ，该策略的前向传播为：

```

x1^(i) = x W1^(i)                                (47)
x2^(i) = x W2^(i)                                (48)
z^(i)  = f(x1^(i)) * x2^(i)                       (49)
y^(i)  = z^(i) W3^(i)                             (50)
y      = all-reduce({y^(i)}_(i=0)^(N_TP-1))       (51)

```

其中 $W1^{(i)}$ 与 $W2^{(i)}$ 的形状为 $(D, D_{FF}/N_{TP})$, $W3^{(i)}$ 的形状为 $(D_{FF}/N_{TP}, D)$ 。

题目 tp_calcs: 张量并行计算 (4 分)

在与 DP、FSDP 相同的设置下, 分析 TP 何时成为通信瓶颈。

(a) 给定形状为 (B, D) 的输入梯度 dy , 写出上述张量并行策略的反向传播。其中 $W1^{(i)}$ 、 $W2^{(i)}$ 的形状为 $(D, D_{FF}/N_{TP})$, $W3^{(i)}$ 的形状为 $(D_{FF}/N_{TP}, D)$ 。

提交内容: 写出一系列描述反向传播的公式。可使用: dy ; 分片权重 $W1^{(i)}$, $W2^{(i)}$, $W3^{(i)}$; 前向传播保存的激活 x , $x1^{(i)}$, $x2^{(i)}$, $z^{(i)}$, $y^{(i)}$; 通信原语; 以及自行定义的中间变量。公式最终应在每台设备上产生 $dW1^{(i)}$, $dW2^{(i)}$, $dW3^{(i)}$ 和反向传播输出 dx 。可以参考 8.2 节的未分片反向传播并进行修改。

(b) 使用 N_{TP} 路 TP 时, 前向传播需要多少 FLOPs? 反向传播呢?

提交内容: 两个用 B 、 D 、 D_{FF} 、 N_{TP} 表示的答案, 并各用一句话说明理由。

(c) 使用 N_{TP} 路 TP 时, 前向传播需要多少通信时间? 反向传播呢?

提交内容: 两个答案, 分别用 B 、 D 、 D_{FF} 、 N_{TP} 、 W 中必要的变量表示, 并各用一句话说明理由。

(d) 固定其他参数后, 在反向传播受到通信瓶颈限制之前, N_{TP} 最大可以是多少? 前向传播呢?

提交内容: 两个不等式, 一边是 N_{TP} , 另一边由 B 、 D 、 D_{FF} 、 C 、 W 中必要的变量构成; 并各用一句话说明理由。

8.5 二维并行 (FSDP + TP)

现在终于可以组合并行策略了。本节研究如何以最佳方式组合 FSDP 与 TP。

提示: 在前几节中, 你应当已经发现, batch size 与模型规模会限制可扩展到多少台设备。增大它们, 可以在通信成为瓶颈之前继续扩展更多设备。但 batch size 超过某个点后, 梯度噪声会显著减小, 失去 SGD 隐式正则化的一部分作用, 性能反而开始下降。这个点通常称为 critical batch size (临界批大小)。另一方面, 缩放定律通常会告诉我们应该选择多大的模型; 这将是下一份作业的任务。

本节采用一个简化场景: 有人已经给定了全部问题参数, 包括 batch size、模型规模、带宽和加速器速度。你的任务是在仍然由计算而非通信主导的前提下, 选择能扩展到最多设备的 FSDP 与 TP 配置。

先看 FSDP 与 TP 的组合机制。每台设备同时有 TP rank $i = 0, \dots, N_{TP}-1$ 和 FSDP rank $j = 0, \dots, N_{FSDP}-1$, 构成一个二维网格, 总设备数为:

$$N = N_{TP} \cdot N_{FSDP}$$

按照 TP, 先把 $W1$ 、 $W2$ 沿输出维分片, 把 $W3$ 沿输入维分片, 因此需要在激活上插入 TP 风格的所有-reduce。

再应用 FSDP: 沿 batch 维对输入分片, 同时把每个权重矩阵沿尚未被 TP 分片的那个维度继续分片。于是, 在执行 TP 风格的前向/反向传播前, 需要对权重做 FSDP 风格的所有-gather; 执行 TP 风格的反向传播后, 还要对权重梯度做 reduce-scatter。

最终, 每台设备 (i, j) 持有以下权重分片:

```
W1^(i,j), W2^(i,j): shape (D/N_FSDP, D_FF/N_TP)
W3^(i,j):          shape (D_FF/N_TP, D/N_FSDP)
```

给定 batch 分片输入 $x^{(j)}$, 其大小为 $(B/N_{FSDP}, D)$, 前向传播为:

```
W1^(i) = all-gather({W1^(i,j)}_(j=0)^(N_FSDP-1))    (52)
W2^(i) = all-gather({W2^(i,j)}_(j=0)^(N_FSDP-1))    (53)
W3^(i) = all-gather({W3^(i,j)}_(j=0)^(N_FSDP-1))    (54)

x1^(i,j) = x^(j) W1^(i)                               (55)
x2^(i,j) = x^(j) W2^(i)                               (56)
z^(i,j)  = f(x1^(i,j)) * x2^(i,j)                     (57)
y^(i,j)  = z^(i,j) W3^(i)                             (58)
y^(j)    = all-reduce({y^(i,j)}_(i=0)^(N_TP-1))      (59)
```

最终得到 batch 分片输出 $y^{(j)}$ ，大小为 $(B/N_{\text{FSDP}}, D)$ 。为简洁起见，本节省略反向传播，只关注前向传播；但到这里，你已经掌握了自己写出反向传播所需的全部信息。

题目 fsdp_tp_calcs: 二维并行计算 (6 分)

在与前述推导相同的设置下，分析二维并行何时遇到瓶颈。

(a) 使用 N_{FSDP} 路 FSDP 加 N_{TP} 路 TP 时，前向传播需要多少 FLOPs?

提交内容： 用 B 、 D 、 D_{FF} 、 N_{FSDP} 和 N_{TP} 表示答案，并用一句话说明理由。

(b) 使用 N_{FSDP} 路 FSDP 加 N_{TP} 路 TP 时，前向传播需要多少通信时间？假设两个轴上的通信可以重叠，即 FSDP 轴的集合通信能与 TP 轴的集合通信同时进行。

提交内容： 用 B 、 D 、 D_{FF} 、 N_{FSDP} 、 N_{TP} 、 W 中必要的变量表示答案，并用一句话说明理由。提示：因为两个轴可以重叠，答案应写成两个量--FSDP 与 TP 集合通信成本--的 $\max$ 。

(c) 在 N_{TP} 与 N_{FSDP} 取最优值时，在前向传播受通信瓶颈限制之前，总设备数 $N = N_{\text{TP}} N_{\text{FSDP}}$ 最大可以是多少？

提交内容： 一个不等式，一边是 N ，另一边由 B 、 D 、 D_{FF} 、 C 、 W 中必要的变量构成；再用若干句话和公式说明理由。

(d) 现在假设 FSDP 轴与 TP 轴的集合通信不能重叠，因为它们共享相同网络资源。在 N_{TP} 与 N_{FSDP} 取最优值时，在前向传播受通信瓶颈限制之前，总设备数 $N = N_{\text{TP}} N_{\text{FSDP}}$ 最大可以是多少？无需把 N_{TP} 与 N_{FSDP} 截断为整数。

提交内容： 一个不等式，一边是 N ，另一边由 B 、 D 、 D_{FF} 、 C 、 W 中必要的变量构成；再用若干句话和公式说明理由。

9 排行榜

作业 2 的排行榜将测试一个 8B 模型完成一次完整训练步骤的速度。请对代码做基准测试，并使用你能想到的技巧优化显存和运行时间。

核心限制是：不能改变模型的输入/输出行为。你的实现将与 `cs336_basics` 目录中的模型对比测试。输入使用 BF16 和因果 mask，并且必须通过与普通实现相同的测试。实现必须由你自己完成，不能使用或复制现有实现。

计时在两块 B200 GPU 上完成，样本 batch size 为 2、sequence length 为 32,768。这个模型是有意设计得很难装入显存。完整配置如下：

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2

cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(
        high=cfg.vocab_size,
        size=(2, cfg.batch_size, cfg.ctx_len),
    )

    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())

    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()

    timing_results = triton.testing.do_bench(
        train_step,
```

```
rep=30_000,
warmup=10_000,
)
print(timing_results)
```

为了自行测试，可以把以毫秒给出的重复时间和预热时间缩短。下面是一些性能与显存优化思路：

- 调整内核的 tile size；可使用 Triton autotune。
- 调整其他 Triton 或 `torch.compile` 配置参数。
- 实现融合 AdamW。
- 基础实现会物化完整 logits，形状为 `[batch, seq_len, vocab_size]`。可以编写一个融合 LM head 与 cross-entropy loss 的内核，也可以让它立即以融合方式完成反向传播。
- 改进 FlashAttention：
- 用 Triton 实现反向传播，而不只是 `torch.compile`；参见 4.2.3 节。
- 反向传播对输入做两次遍历：一次计算 `dQ`，另一次计算 `dK` 和 `dV`，从而避免原子操作或线程块间同步。
- 使用因果 mask 时，让确定全为 0 的 tile 对应的程序实例提前终止。
- 把完全不被 mask 的 tile 与对角 tile 分开：前者完全不比较索引，后者只比较一次。
- 在 Hopper 之后的架构上，按照本作业教程的类似模式使用 TMA（Tensor Memory Accelerator）。
- 只有确实需要节省显存时，才使用激活检查点以运行时间换显存。

课程组会验证排名前 5-10 的提交是否正确并核实性能。实际用于计时的测试就是上面的代码。

题目 leaderboard：排行榜--最快训练步骤（10 分）

基准测试将在两块 B200 GPU 上，以 batch size 2 运行。根据完成一个完整训练步骤的墙上时钟时间评分，包括前向传播、损失、反向传播和 AdamW 更新。

从空的 PyTorch/Triton 缓存开始，整个基准测试必须在 10 分钟内完成，因此要谨慎使用过度激进的 `torch.compile` 和 Triton autotune。

提交内容： 使用 AdamW 完成一次完整前向加反向训练步骤的最佳墙上时钟时间。

课程组期望排行榜提交能超过 10 秒的朴素基线。

在这里提交排行榜结果：

github.com/stanford-cs336/assignment2-systems-leaderboard

参考文献

1. T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” 在线地址：
<https://arxiv.org/abs/2307.08691>
2. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” Advances in Neural Information Processing Systems, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. 在线地址：<https://openreview.net/forum?id=H4DqfPSibmx>
3. M. Milakov and N. Gimselshein, “Online normalizer calculation for softmax.” 在线地址：
<https://arxiv.org/abs/1805.02867>
4. H. He, “Making Deep Learning Go Brrrr From First Principles,” 2022. 在线地址：
https://horace.io/brrr_intro.html

5. S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.” 2020.
6. J. Austin et al., “How to Scale Your Model,” 2025.
7. H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, “The Ultra-Scale Playbook: Training LLMs on GPU Clusters.” 2025.